中图分类号：TN384 密　　级：公　开

济南大学

本科毕业设计



基于 Unity 的类恶魔城游戏的设计与实现基于 Unity 的类恶魔城游戏的设计与实现

学　　院　　信息科学与技术学院
专　　业　　计算机科学与技术
班　　级　　计科 2202
学　　生　　葛广一
学　　号　　202231222052
指导教师　　赵亚欧

二〇二 六 年 五 月　　日

济南大学毕业论文（设计）诚信承诺书

本人声明所提交的毕业论文（设计）是本人在导师指导下独立完成的研究成果，本论文中不包含他人已经发表或撰写过的研究成果，也不包含为获得湖南科技大学或其它教育机构的学位或证书而使用过的材料。对本研究做出重要贡献的个人和集体，均已在文中以明确方式标明。除文中已注明引用的内容外，本论文不包含其他人已经发表或者撰写过的研究成果，也不包含为获得济南大学或者其他教育机构学位或证书而使用过的材料。本研究工作同我一起工作的同志对于所做的任何贡献，在论文中都已作出了明确的说明，并表示了谢意。

学生签名：　　日期：　　年　　月　　日

济南大学毕业论文（设计）使用授权书

本人同意学校保存本论文（设计），同意学校向国家有关部门或机构送交本论文（设计）的复印件和电子版，允许论文被查阅和借阅。本人授权济南大学可以将

本论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存论文（设计）。本人电子文档的内容和纸质论文的内容相一致。除保密期满的保密论文之外，允许论文被查阅、借阅，可以公开（刊登）论文的全部或者部分内容。该公布（包括刊登）的授权，由济南大学教务处办理。

学生签名：　　导师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　要

本文以 Unity 平台下 2D 横版动作游戏 Demo 为研究对象，主要目的就是完成一款类恶魔城风格的小型可运行作品，项目使用 C#脚本来编写功能，用有限状态机组织玩家、普通敌人和 Boss 的行为流程；角色移动、跳跃、冲刺、普通攻击、弹反、技能释放等主要依靠 Rigidbody2D、碰撞检测和 Animator 之间的配合来实现，战斗部分包含生命值管理、伤害判定、受击反馈、飞剑、黑洞、Boss 二阶段激光等内容，关卡部分加入了落石机关、骷髅王和 Tilemap 通路事件，玩家会先经历基础战斗，再过渡到最终 Boss 战，系统还配有开始、暂停、死亡、通关菜单以及血条、音效和 Windows 打包支持，测试结果表明，该 Demo 可以完成从开始、探索、战斗、机关躲避、Boss 挑战、结算这一整条流程，可以作为毕业设计展示和论文分析的实践基础。

关键词：Unity；C#；2D 动作游戏；有限状态机；Boss 战

ABSTRACT

The Development Object of this thesis is a 2D Metroidvania-style action game demo. Unity and C# were used to develop the game. A finite state machine is used to define the script boundaries of different action phases for the player, general enemies and bosses more clearly. Rigidbody2D, Collider2D, Animator and Tilemap are used together with custom scripts to control movement, jumping, dashing, attacks, skills, terrain collision, etc. The functions of the combat module are to manage health, detect damage, provide knockback feedback, throw a sword and control a black hole, etc., and there is also a two-phase boss battle with laser warnings. There is a demo, and there will be falling stone traps, an improved Skeleton King enemy, and a Tilemap gate that opens after defeating the Skeleton King. In addition to a menu, a health bar, sound effects and a window builder have also been added; now, the whole play process of the graduation design project has been realised.

Key words: Unity; C#; 2D action game; finite state machine; boss fight

目　录

摘　　要

ABSTRACT

1　绪　论

1.1　选题背景与意义

1.2　国内外研究现状

1.3　本文主要工作

1.4　论文组织结构

2　需求分析

2.1　系统目标

2.2　功能需求分析

2.2.1　玩家控制需求

2.2.2　战斗与技能需求

2.2.3　敌人与 Boss 需求

2.2.4　UI 与流程需求

2.2.5　关卡机关与流程解锁需求

2.3　非功能需求分析

2.4　可行性分析

3　系统总体设计

3.1　开发环境与技术选型

3.2　系统总体架构

3.3　核心模块划分

3.4　状态机设计

3.5　资源与场景组织设计

3.6　关卡流程与交互事件设计

4　系统详细设计与实现

4.1　玩家控制系统实现

4.2　角色属性与伤害判定实现

4.3　技能系统实现

4.4　普通敌人 AI 实现

4.5　关卡机关与精英敌人实现

4.6　Boss 战斗系统实现

4.7　UI 与音效系统实现

5　系统测试

5.1 测试环境与测试方法
5.2 功能测试
5.3 典型问题与优化
5.4 测试结果分析
结论
参考文献
致谢

1 绪论

1.1 选题背景与意义

随着数字娱乐产业的发展，独立游戏给 2D 横版动作游戏留有较大的创作空间，与大型 3D 游戏相比，2D 游戏在美术资源、场景大小、运行成本上更容易控制，但是操作手感、碰撞反馈、敌人行为、关卡节奏不会因为画面维度的减少而变得简单；类恶魔城游戏一般会包含横版探索、动作战斗、能力成长、Boss 挑战，玩家的体验既来自于角色本身的操作，也来自于敌人反馈、地图结构和流程安排，基于 Unity 完成一个规模不大但流程比较完整的 Demo，既可以展示游戏开发中所具备的综合能力，也比较适合用于计算机专业毕业设计实践。

从软件工程的角度来说，本课题不是把素材直接摆到场景里，也不是只完成一个角色移动脚本就结束，输入响应、物理控制、动画切换、敌人 AI、技能系统、UI 交互、资源管理都需要放在同一个稳定的流程中；动作游戏中的状态变化比较频繁，如果结构没有提前理清楚，代码中就会出现条件交叉、动画混乱、后续扩展困难等问题，有限状态机把角色行为拆分成几个阶段，移动、攻击、受击、死亡分别处理，代码之间的耦合会低一些，阅读、排查、调试也更容易。

课题实践以 Unity2D 项目为载体，以能运行、能演示、也能将实现过程说明白的毕业 Demo 为目标，项目以玩家控制、普通敌人、Boss 战、技能、血量 UI 和菜单

流程为基本玩法闭环，为论文中需求分析、总体设计、详细实现和测试验证提供工程依据；通过该项目的训练，使毕业设计从文字说明变成有实际运行作品的毕业设计。

1.2 国内外研究现状

Unity 是一款 2D 和 3D 游戏开发时常用的工具，提供了 Tilemap、Animator、Rigidbody2D、Collider2D、AudioSource、Canvas 等很多成熟的组件，可以满足横版动作游戏从关卡搭建、角色控制、动画播放、UI 展示等各种需求，很多独立游戏就是利用 Unity 的组件化系统快速搭建起原型，再用脚本来调整输入、物理参数、状态切换，从而得到更加符合自己想要的玩法的体验。

在游戏 AI 的设计中，有限状态机、行为树、分层状态机等都是比较常见的组织方式，拿毕业设计规模的 2D 动作游戏 Demo 来说，有限状态机的实现成本低、结构直观、调试时容易看出对象当前处在哪个阶段；普通敌人用待机、巡逻、追踪、攻击、受击、死亡等状态来描述行为，Boss 可以在普通敌人基础上加上传送、施法、二阶段转场等状态来表现更复杂的战斗节奏；与把全部逻辑堆在一个 Update 函数中的方式相比，状态机可以让每一个状态只负责本阶段要做的事情，后续维护会轻一些。

手感一般同输入响应速度、移动加速度、跳跃高度、碰撞检测稳定度、攻击前后摇、受击反馈等有着联系，本项目并没有试图构建起一个商业作品复杂的程度，

而把重心放在了毕业 Demo 可以展现的核心体验部分上，即玩家能够较为流畅地移动、跳跃、冲刺、攻击，敌人可以在一定的范围内找到玩家并做出反应，Boss 会给出阶段变化以及明显的攻击提示；这样的取舍有利于在较短的开发周期内得到稳定的、可以答辩的系统成果。

1.3 本文主要工作

本文主要对基于 Unity 的 2D 类恶魔城动作游戏 Demo 进行设计和实现，大致可以分为如下几个方面。

一是完成玩家控制系统的实现，项目使用 Unity 输入、Rigidbody2D、Collider2D、Animator 来实现玩家的移动、跳跃、冲刺、墙面交互、普通攻击、飞剑技能、黑洞技能等动作，并将这些动作放入到玩家的状态机中进行统一管理，使状态的切换有比较清晰的界限。

二是完成敌人和 Boss 战斗系统的创建，普通敌人以骷髅敌人为主，巡逻、发现玩家、追踪、攻击、受击、死亡等行为，Boss 以 DeathBringer 为主，近战攻击、法术攻击、传送、二阶段激光转场、死亡反馈等流程。

三是完成战斗判定和角色属性系统的开发，项目中 CharacterStats、PlayerStats、EnemyStats 等脚本被用来管理最大生命值、当前生命值、伤害、无敌时间、死亡状态，攻击判定则是通过碰撞检测、触发器、动画事件之间的配合来完成的。

第四章是 Demo 交互与发布支持，本系统中有血条、敌人血条、boss 血条、开

始菜单、暂停菜单、死亡菜单、过关菜单、基础音效管理、打包 Windows 应用程序等功能。

五是完成关卡机关和流程解锁的设计，在主线路径上加入落石机关、骷髅王精英敌人、击败后打开通道的 Tilemap 门，玩家在进入最终 Boss 区域之前会经历探索、躲避、战斗、解锁等环节，使 Demo 的流程更加完整一些。

1.4 论文组织结构

本文共分为六个主要部分，在具体内容的安排上，本文共分为六个部分，分别是绪论、需求分析、系统总体设计、系统具体实现、系统测试方案及运行结果、结论。

2 需求分析

2.1 系统目标

本系统的主要目的就是实现一个可以独立运行、现场演示的 2D 类恶魔城动作游戏 Demo，Demo 不追求覆盖完整的商业游戏所有内容，只是保证核心玩法闭环能够走通，即玩家进入游戏后可以控制角色完成基本的探索，与普通敌人战斗，最终战胜 Boss 并得到通关结果；系统要在有限的开发周期内保持功能边界清晰、运行状态稳定、结构也便于说明，才能支撑毕业论文撰写和答辩演示。

按照毕业设计任务书及目前项目情况，系统目标可总结为三点，一是具有较为良好的玩家控制与战斗反馈，二是以有限状态机为基础的普通敌人和 Boss 行为逻辑，三是菜单、血量显示、音效、打包发布等 Demo 支持功能，这些目标一起构成了本文分析及作品展示的主要依据。

2.2 功能需求分析

从游戏 Demo 运行流程上看，系统功能可以分为玩家控制、战斗技能、敌人 AI、关卡机关、Boss 战、UI 交互、音效发布等模块，各个模块的需求如下表 2.1 所示。

表 2.1 系统主要功能需求

模块	功能需求	说明
玩家控制	移动、跳跃、冲刺、墙面交互、普通攻击、弹反	保证角色能够完成横版动作游戏的基础操作与近战反制

技能系统	Dash、Sword、Blackhole	飞剑支持投掷、插入、召回和接剑，黑洞支持范围控制与持续伤害
普通敌人	巡逻、发现、追踪、攻击、受击、死亡	通过状态机组织骷髅敌人的行为流程
Boss 战	近战、施法、传送、二阶段激光、死亡	提供阶段变化和较完整的战斗演示
UI 系统	开始、暂停、死亡、通关、血条	保证游戏流程可开始、暂停、重开和结束
音效与发布	攻击、受击、死亡、菜单等音效，Windows 打包	增强反馈并支持脱离编辑器运行
关卡机关	落石陷阱、精英怪、击败后开门	通过场景事件提高探索与战斗之间的衔接

2.2.1 玩家控制需求

玩家角色是系统交互的中心，玩家角色可以根据键盘输入进行左右移动、跳跃、冲刺、墙面滑落、墙跳、普通攻击、Q 键弹反，在动画状态下表现出相应的动作；为了减少输入和动画之间的相互干扰，玩家的行为不能全部堆在一起，而应该通过状态机将其拆分成待机、移动、跳跃、空中、冲刺、攻击、弹反、瞄准飞剑、接剑等状态。

玩家控制还需要一定的容错性、反馈性，在攻击的过程中会产生相应的攻击判定，被击中之后会扣除生命值并且触发反馈，死亡后会进入死亡表现并弹出死亡菜单；飞剑、黑洞技能需要玩家输入触发，与玩家状态切换一致，不能破坏原有的动作流程。

2.2.2 战斗与技能需求

战斗系统要支持玩家通过普通攻击、飞剑和黑洞对敌人造成伤害，也要支持敌

人和 Boss 对玩家造成伤害，伤害不能只是数值的变化，还要联动受击反馈、击退方向、无敌时间、死亡判定和音效播放；Q 键弹反在目前的 Demo 中和普通攻击同属于基本近战交互，但是只针对开放反制窗口的骷髅类敌人，Boss 攻击不在其范围内。毕业 Demo 战斗系统的目的是判定要清楚、反馈要明显、流程要稳定。

技能部分飞剑技能有瞄准轨迹、投掷、命中插入、召回和接剑这五种过程，黑洞技能有生成、范围显示、冻结敌人、持续造成多次伤害、销毁这五种过程，技能实现尽量复用统一的 Skill 基类和 SkillManager 管理方式，把不同技能之间的耦合控制在较低的程度。

2.2.3 敌人与 Boss 需求

普通敌人在没有被发现玩家的时候会做基础巡逻或者等待，在发现玩家之后会进入追踪或者攻击状态，当玩家离开检测范围后，敌人会在一段时间之后退出战斗状态，减少全图追踪或者坐标重叠抖动等不自然现象；敌人受到攻击时会产生正确的击退方向，在生命值归零时也会播放正确的死亡表现。

Boss 战是 Demo 里最重要的展示内容，Boss 要能和普通敌人有不一样的状态变化，近身战、近战攻击、施法攻击、传送、二阶段转场、死亡；二阶段转场有明显的提示，比如激光攻击，可以用瞄准线和延迟发射给玩家留出反应时间，玩家也可以用冲刺来躲避，使战斗节奏和演示效果更清楚。

2.2.4 UI 与流程需求

游戏流程从开始菜单进入游戏，运行时按下 Esc 可以暂停，给出继续、重新开始、退出三个选项，玩家死亡后暂停游戏并显示重新开始或者退出按钮，Boss 被击败后显示通关界面；Demo 要有明显的开始和结束，血条 UI 要能随着角色生命值的变化实时更新，Boss 血条也要方便玩家观察战斗进程。

对于目前的 Demo 来说，音效系统要对攻击、受击、死亡、技能、Boss 激光以及菜单点击等操作给出基本的反馈，虽然音效并不是系统的核心算法，但是会对演示时的完成度和操作反馈感产生较大的影响，因此将其作为毕业 Demo 的辅助功能来实现。

2.2.5 关卡机关与流程解锁需求

为了使 Demo 不直接进入 Boss 战，在普通敌人上增加一些可以解释清楚的环境机关，这些机关不会做复杂的解谜，但是会起到节奏调节和演示扩展的作用；玩家在移动的时候要注意观察场景、躲避落石，在打败精英敌人之后打开接下来的道路，使角色控制、敌人战斗、场景交互都在同一个演示路径上出现。

落石机关在玩家进入检测区之后才开始触发，每一个石块都要单独判断，不能因为一个触发器而使整排石块一起落下；下落阶段设置加速度和较高速度，产生需要冲刺躲避的压力，回收阶段速度较慢，机关可以反复使用，石块碰到玩家时直接触发死亡，危险结果比较明确。

精英骷髅王是连接机关区和后面通道的连接件，它继承了普通骷髅敌人的状态

机和攻击逻辑，但是通过更高的生命值、更大的体型和头顶光环与普通敌人区别开来；击败骷髅王之后，场景右侧柱子底部的通路应该被解除阻挡，玩家可以继续前进，该需求没有引入完整的钥匙背包系统，而是通过敌人死亡事件直接控制 Tilemap 通路，比较符合目前 Demo 的范围。

需求边界在这个阶段也要写清楚。类恶魔城游戏经常会加入地图回环、能力门槛、道具成长、存档系统等，但是这些都会把开发重心放到更大的关卡框架上。目前作品的目标集中在一条可以完整演示的路线之上，即先让玩家完成移动、战斗，再经过机关、精英敌人，最后到达 Boss 区域。这样的处理不是系统设计的减少，而是在已经存在的交互链上，将需求集中到可以验证的 Demo 中已经存在的交互链上，使功能说明和实际运行结果一致。

从验收的角度来说，功能是否存在不能只看脚本文件的数量，还要看它能否在局程中被玩家自然地使用。开始菜单可以进入游戏，角色可以到达机关区域，落石会对冲刺躲避造成压力，骷髅王死亡之后通路要确实打开，Boss 战还要给出失败或者通关的反馈。只有当一个节点发生中断的时候，后面的演示才会被截断，所以本设计把流程连通性作为需求分析中的一项重要约束，而不是把各个模块孤立地列出。

2.3 非功能需求分析

系统非功能需求大体上包括运行稳定、便于维护、现场能演示、后续可以继续增加内容等几个方面，稳定性上 Demo 运行时不能出现阻塞流程的空引用异常、角

色卡死、菜单无法响应、Boss 无法结束等情况，可维护性上玩家、敌人、Boss、技能、属性和 UI 应尽量分布在独立脚本和目录里，便于后续查找和修改。

从可演示性来说，项目需要能够在 Windows 下打包成可运行程序，在答辩或者展示的时候不会依赖于 Unity 编辑器；从可扩展性来说，虽然目前还没有实现完整的背包、复杂的存档和大型非线性地图，但是系统应该保留继续添加新技能、新敌人、新关卡和存档的功能结构基础。

2.4 可行性分析

技术层面上，Unity 2022.3 LTS 已经提供较完整的 2D 开发支持，Tilemap 能够搭建关卡，Rigidbody2D 和 Collider2D 能够处理移动与碰撞，Animator 用于控制序列帧动画，Canvas 负责菜单和血条 UI，C#支持面向对象封装，适合编写 Entity、CharacterStats、Skill 和 State 等基础类。

从时间安排上来说，项目把范围限定在毕业 Demo 而不是完整的商业游戏，先完成玩家控制、普通敌人、Boss 战、基础 UI 等价值较高的部分，从而降低开发风险；从成本上来说，项目开发主要使用 Unity 个人版、公开课程资源、本地开发环境，不需要额外的服务器和商业部署成本，因此本课题在毕业设计周期内具有较好的实现可行性。

3 系统总体设计

3.1 开发环境与技术选型

本设计使用 Unity 引擎和 C#语言进行开发，其主要的开发环境和工具见表 3.1[8-9，15]。

表 3.1 开发环境与技术选型

类别	工具或技术	用途
游戏引擎	Unity 2022.3 LTS	场景搭建、物理系统、动画、UI 和打包发布
开发语言	C#	编写玩家、敌人、技能、UI 和管理器脚本
物理组件	Rigidbody2D、Collider2D	处理角色移动、碰撞和触发检测
关卡工具	Tilemap	绘制横版地形、背景和碰撞区域
动画系统	Animator、Animation Clip	管理角色动作、攻击和死亡动画
版本管理	Git、GitHub	保存项目版本并支持回退与同步
发布平台	Windows	生成可直接运行的 Demo 程序

在技术选择上，Unity 可以采用组件化的组织方式来管理 GameObject 和脚本，非常适合毕业设计中快速搭建可运行的作品，C#脚本用来描述各个模块的行为，对玩家状态切换、敌人检测、Boss 攻击、技能生成、血量更新、菜单响应等都可以通过脚本和 Unity 组件一起实现；与从底层图形库开始实现相比，使用 Unity 可以让开发的重点更多地放在玩法逻辑和软件结构上，更符合本课题的目标[8-9]。

3.2 系统总体架构

从运行过程上看，系统可以看作是由场景对象层、组件行为层、状态控制层、关卡事件层、界面反馈层共同构成的，场景对象层用来存储 Tilemap、玩家、敌人、Boss、摄像机、Canvas 等可见对象，组件行为层用 MonoBehaviour 脚本来把输入、物理、动画、碰撞、音频等联系起来，状态控制层把角色在各个阶段的行为拆分成明确的状态，界面反馈层完成血量、菜单、通关提示等展示的任务，这样的划分使得项目虽然规模不大，但是仍然保持了比较清晰的软件结构。

玩家输入不会直接修改所有的对象状态，而是先驱动玩家状态机，再由状态机调用移动、攻击或者技能相关的函数；敌人和 Boss 也不是在 Update 里堆砌很多条件判断，而是依靠各自的状态类去判定追踪、攻击、受击、死亡等行为，UI 模块则是靠角色属性事件或者流程控制脚本来被动更新，从而减小了界面逻辑反向侵入战斗逻辑的可能性，进而降低了脚本之间的强依赖性，并且便于在答辩演示时解释系统的运行链路。

系统整体为分层、模块化结构，底层为 Unity 场景、Tilemap、动画、碰撞体、音频资源、预制体，脚本层按功能分为玩家控制、敌人 AI、Boss 逻辑、技能系统、角色属性、UI 系统、音效管理，运行层用 Unity 生命周期函数和动画事件驱动状态更新、碰撞判定、界面刷新。

从对象组织上来说，玩家、普通敌人、Boss 会继承或者复用 Entity、

CharacterStats 等基本能力，玩家、敌人的复杂行为由状态机来管理，每一个状态都负责自身进入、更新、退出的逻辑；技能模块使用 SkillManager 来获取所有的技能组件，减少了玩家脚本直接管理所有的技能细节，UI 模块监听角色生命值的变化以及游戏流程事件，然后进行菜单的显示和血条的显示。

各个模块之间不是完全独立的。玩家输入进入状态机之后，就会调用 Entity 的速度、碰撞、受击能力，战斗结果再落到 CharacterStats 的生命值变化上；血量变化之后，UI 脚本就会通过事件刷新显示，如果生命值归零，还会继续影响死亡菜单、通关菜单或者关卡门状态。也就是说项目表面上是角色移动、敌人追踪、界面变化，实质上是一条从输入、状态、判定、属性到反馈的执行链，这条执行链就是总体架构要说明的重点。

项目中对象的创建方式也有所区别。飞剑是需要反复使用并且配置碰撞、刚体、控制器的攻击对象，所以用 Prefab 为主；Boss 法术也是用预制体来保存表现和判定。瞄准点、部分黑洞运行时能自动创建出来，如果没有另外配置的话，脚本也会产生可见的对象并执行完整的技能流程。两种方法各有利弊，预制体可以保持较好的对象结构，运行时产生比较小的提示和临时效果，混合使用可以使 Demo 对于大量的手工场景有更少的依赖。

3.3 核心模块划分

根据目前的项目代码结构，系统主要模块划分如下表 3.2 所示。

表 3.2 系统核心模块划分

模块	代表脚本或目录	主要职责
玩家模块	Scripts/Player	玩家状态机、移动、跳跃、冲刺、攻击、技能触发
敌人模块	Scripts/Enemy/Skeleton	普通敌人的巡逻、追踪、攻击、受击和死亡
Boss 模块	Scripts/Enemy/DeathBringer	Boss 状态切换、传送、施法、二阶段激光和死亡
技能模块	Scripts/Skills	Dash、Sword、Blackhole 等技能创建、冷却和控制
属性模块	Scripts/Stats	生命值、伤害、无敌时间和死亡判定
UI 模块	Scripts/UI	血条、开始菜单、暂停菜单、死亡菜单和通关菜单
管理模块	Scripts/Managers	音效播放、玩家引用和全局辅助管理
机关模块	Scripts/Traps	落石触发、石块回收、骷髅王死亡后 Tilemap 通路解锁

这样模块划分使得论文中详细的实现可以按功能展开，玩家控制系统的实现可以围绕 PlayerStateMachine 以及各种 PlayerState 来说明，敌人 AI 的实现可以围绕 EnemyStateMachine 以及 Skeleton 状态来说明，关卡机关的实现可以围绕 FallingStoneTrap 和 SkeletonKingTileGate 来说明，Boss 战的实现可以围绕 DeathBringer 各个状态和激光控制器来说明；各个模块之间通过明确的引用和事件调用来协作，减少单个脚本承担过多职责的情况。

3.4 状态机设计

本设计中状态机的主要目的就是使每一个状态只处理当前阶段最核心的输入和

行为，玩家控制中待机状态检测移动、跳跃、攻击输入，移动状态控制水平速度和移动动画，空中状态控制下落、落地、墙面检测，冲刺状态在短时间内屏蔽普通移动并施加冲刺速度；这样玩家在同一时间只处于一个主要的动作状态之中，可以减少多种输入同时触发造成的冲突。

敌人和 Boss 状态机除了完成动画的切换之外，还需要对 AI 进行节奏上的控制，普通敌人根据玩家的位置、检测结果和 battleTime 决定是否继续追踪或者退出战斗，Boss 战斗的状态会加入攻击冷却、法术释放、传送、二阶段的判断，使得状态机不是为了复杂的 AI 算法而存在，而是为了达到稳定、可控、可解释的目的，使 Demo 可以呈现较为完整的战斗过程。

有限状态机是本系统中比较重要的一个结构，它的基本思想就是把一个对象运行时的行为拆分成若干个互斥或者阶段性状态，在满足条件的时候从当前状态切换到目标状态；每个状态一般包含 Enter、Update 和 Exit 等逻辑，其中 Enter 用来初始化状态，Update 用来持续检测输入或者环境条件，Exit 用来恢复或者清理状态。

玩家状态机主要是为了解决动作频繁切换的问题，玩家可以处于待机、移动、跳跃、空中、冲刺、普通攻击、飞剑瞄准、接剑、黑洞释放等状态，如果把所有的逻辑都写在 Update 函数里，那么不同的输入就会互相影响；用状态机拆分之后，攻击状态只处理攻击期间的行为，冲刺状态只处理冲刺速度和结束条件，接剑状态只处理接剑动画和后续切换，代码的含义会更加清晰。

敌人和 Boss 状态机主要是控制 AI 的行为，普通敌人根据检测范围、玩家位置、攻击距离和战斗时间在巡逻、追踪、攻击、受击、死亡之间切换，Boss 在普通敌人的基础上增加了施法、传送和二阶段转场；二阶段转场用激光预警和延迟发射来制造节奏的变化，使 Boss 战不只是简单的追逐和近战攻击。

状态切换还要和 Unity 本身的执行方式相配合。持续检测应该放在更新里面，玩家是否按下移动键、敌人是否还在追踪时间内都可以由它来处理，有明确时点的行为就交给动画事件或者协程去处理，比如攻击动画某帧判定、落石经过预警后进入下落、Boss 激光在锁定目标后延迟发射。将这些职责分离开来之后，动作的表现以及逻辑的时机就更加容易对应起来，同时也可以减少每一个状态所要承担的太多计时细节。

状态机不能完成所有的工作。将伤害计算、音效加载、血条更新、关卡开门等都放在角色状态类中，那么状态数量虽然清楚，但是类之间还是会重新变乱。本设计把状态机更多的应用到了组织行为阶段，把属性变化交给了 CharacterStats，把通用受击和速度接口放在了 Entity 上，把界面流程交给了 UI 脚本，把音效播放交给了 AudioManager。因此边界更接近于目前 Demo 所要求的结构，也便于在论文中分别说明。

3.5 资源与场景组织设计

项目按照功能和用途把资源分为 Player、Enemy、Skills、Stats、UI 五个目录，

分别存放玩家脚本、普通敌人和 Boss 脚本、技能脚本、属性脚本、界面脚本，动画控制器、图片、音效和预制体也按类型分类；当飞剑技能出现空引用时，可以从 SkillManager、Sword_Skill、飞剑 Prefab 这三个地方依次查找。

Demo 采用线性的演示流程，玩家出生之后先接触到基础移动以及普通敌人，接着经过落石机关和骷髅王区域之后，最后到达 Boss 战区域；这样的安排并不是为了构建完整的大型地图，而是为了在答辩现场尽可能快地展示出玩家控制、敌人 AI、技能释放、Boss 阶段切换、菜单流程，场景中碰撞层次进行了区分，地面和墙体用来阻挡角色，背景层只起到视觉表现的作用，从而减少背景装饰对物理逻辑的影响。

就当前 Demo 而言，项目资源主要放置在 Assets 目录下，脚本、动画、图像、预制体、音效和场景分别按照用途组织；Scripts 目录按照 Player、Enemy、Skills、Stats、UI 和 Managers 等子目录拆分，Prefabs 目录用来保存飞剑、Boss 法术、敌人和 UI 相关预制体等可复用对象，Resources/Audio/SFX 目录则用于存放运行时由 AudioManager 加载的音效资源。

场景设计以 SampleScene 为主，地面、平台、背景层次用 Tilemap 搭建，玩家、普通敌人、Boss、相机、UI Canvas 在一个场景中协同工作；为了使演示过程简单，场景主要包含基础探索、普通敌人战斗、落石机关、精英敌人和 Boss 区域，目前没有扩展大型地图和完整的存档流程，这样可以减轻调试负担，也可以使论文内容与

实际项目保持一致。

场景组织要考虑到对象的层次以及是否可以编辑。地面、墙体、平台用 Tilemap 来完成，背景层只起到视觉展示的作用，玩家、普通敌人、骷髅王、Boss 都是可以单独配置的场景对象；机关脚本挂在相应的石块或者关卡对象上，参数可以在 Inspector 中进行修改。因此，当在调整落石距离、Boss 触发距离或者骷髅王的位置时，并不会造成整个地图逻辑的改变，场景编辑和脚本维护之间存在着比较明显的关系分工。

镜头与场景有界限的时候就出现了有关卡。目前 Demo 使用相机跟随的方式让玩家处在画面的视野之内，平台高度、落石位置以及 Boss 区域距离都要根据可见范围来判断，如果镜头过低，画面会出现不合适背景边界的情况；如果 Boss 放得太近，玩家在前一段流程中就会进入最终战斗。因此场景设计不能只是放置素材，还要使相机、碰撞、敌人检测、演示路线这些配合使用起来。

本章将按照前面总体设计的要求来具体说明各个模块的实现，即玩家状态机、技能控制器、敌人检测逻辑、Boss 二阶段激光、血量 UI 以及菜单流程等。

3.6 关卡流程与交互事件设计

关卡流程以基础操作展示、普通敌人战斗、机关躲避、精英敌人解锁、Boss 战为线性结构，在较短的演示时间内可以覆盖项目大部分核心模块，也能使玩家逐渐接触到不同的交互方式，如果直接把玩家放在 Boss 面前，虽然可以展示 Boss 战，

但是玩家控制、普通敌人 AI、场景机关等内容很难自然地展现出来，加入中间区域之后，Demo 过程会更加完整。

关卡事件并没有建立复杂的任务系统，目前的做法是使用 Unity 场景对象和脚本事件，玩家进入落石下方时，传感器调用对应石块的 Trigger 方法，石块下落、停留、回收结束后，会重新进入可触发状态；骷髅王死亡时，CharacterStats 触发 onHealthChanged 事件，机关门脚本检测到生命值归零后移除指定 Tilemap 格子，原本被柱子挡住的位置随后就可以通行。

优点是成本低，与现有的角色属性和碰撞系统一致，机关区域主要用 Collider2D、Trigger、Coroutine 来实现，解锁区域主要用 Tilemap 和角色死亡事件来实现，不需要引入背包、任务、存档等比较重的系统，对于毕业 Demo 来说，这样的做法既能表现出关卡互动，又能控制开发风险。

表 3.3 关卡流程与触发事件设计

流程阶段	触发条件	系统反馈
出生与基础移动	玩家点击开始游戏	恢复时间流动，玩家进入待机或移动状态
普通敌人区域	玩家靠近或攻击骷髅敌人	敌人进入战斗状态，刷新 battleTime 并追踪玩家
落石机关区域	玩家进入石块正下方传感器	对应石块预警、加速下落、造成即死判定并回收
骷髅王区域	玩家接近或攻击精英骷髅	骷髅王以更高血量和体型进行战斗
通路解锁	骷髅王生命值归零	指定 Tilemap 砖块被移除，柱子底部变为可通行
Boss 区域	玩家进入 Boss 战范围	Boss 启动战斗流程并进入阶段判断

关卡流程要兼顾演示顺序和玩家的学习成本，如果玩家一进入 Demo 就直接面

对 Boss，那么由于对移动、冲刺、攻击节奏不熟悉而失败的情况就会发生，同时普通敌人和技能系统的体现也会被忽略，因此关卡中会先出现普通骷髅、落石机关、骷髅王等，从而逐步让玩家熟悉操作，普通骷髅用来检验攻击和受击的反应，落石机关需要玩家用移动和冲刺，骷髅王把基本战斗提升到小型精英战，这样的顺序不需要额外的文字教学，玩家依靠场景内容本身就能理解下一步的目标。

从工程实现的角度来说，关卡事件层被控制在比较轻量的范围内，项目没有引入完整的任务系统、事件总线或者存档驱动的关卡状态，而是用现有的 CharacterStats 事件、Collider2D 触发器、Tilemap 接口来完成局部事件；该选择与毕业 Demo 的目标相契合，一方面可以展示“击败敌人后改变场景”的流程逻辑，另一方面也不会因为系统过大而造成调试成本的上升，如果后续要扩展成正式游戏，可以在此基础上抽象出 GameEvent 或者 LevelEvent 管理器，将落石、门锁、钥匙、检查点等都纳入到事件管理当中。

关卡中可以通行和不可以通行的区域由 TilemapCollider2D 来处理，这样地图编辑和碰撞生成就统一了，绘制砖块就会产生阻挡效果，移除砖块的时候也会同时解除碰撞，骷髅王通路解锁就是利用这一点实现的，脚本不直接移动墙体，也不单独关闭某个 BoxCollider2D，而是移除 Ground Tilemap 中的具体 cell，由于视觉和物理来源一致，玩家看到砖块消失后就可以通过，减少了“看起来能走但被挡住”或者“看起来有墙但能穿过的”不一致的问题。

落石机关和骷髅王两种关卡节奏不一样，前者是空间和时间的判断题，后者是战斗操作题，落石机关只需要观察石块位置及时冲刺即可通关，而骷髅王需要多次攻击、躲避、观察血条才能通关；两种内容都复用了已有的系统，但带来了不同的体验，落石复用了碰撞和生命值，骷髅王复用了敌人状态机和血条，复用的方式在较少新增代码的基础上增加了流程层次，也是小规模 Demo 扩展内容时比较稳妥的做法。

4 系统详细设计与实现

4.1 玩家控制系统实现

玩家移动使用 Rigidbody2D 和自定义速度设置方法，水平移动时脚本根据输入的方向来设置刚体的横向速度，用动画参数表现移动的状态，跳跃时脚本会调整竖直方向的速度，使角色脱离地面，冲刺状态会在短时间内覆盖普通移动的逻辑，让角色快速向指定的方向移动；由于这些行为都会改变刚体的速度，所以项目把速度设置集中到 Entity 给出的方法上，减少不同状态对刚体的重复操作。

墙面交互属于横版动作游戏中常见的问题，在设计中用 GroundCheck、WallCheck 等检测点检测玩家和地面或者墙壁的接触情况，当玩家进行滑行或者跳墙时，对速度进行控制或者改变，以避免玩家在原本轨迹上下滑落在墙上，使墙壁的互动更加接近横版动作类手柄游戏的操作方式。虽然 Demo 并没有达到复杂的能力解锁的效果，但是墙面的交互能体现出角色控制系统拓展的特点。

输入屏蔽机制看似简单，但是会对演示体验产生影响，开始菜单被点击之后，鼠标输入会继续传递到游戏中，如果没有做处理，玩家进入场景的时候就会直接打出一次普通攻击；项目使用 BlockActionInputFor 对攻击和技能输入进行短暂屏蔽，将菜单点击和战斗输入分开，在 Demo 开始阶段也会显得更加自然。

玩家控制系统是本设计的基础模块，Player 脚本继承自 Entity，在 Awake 方法中

创建 PlayerStateMachine，并依次实例化 idleState、moveState、jumpState、airState、dashState、wallSlideState、wallJumpState、primaryAttackState、aimSwordState、catchSwordState 和 blackholeState 等状态；Start 方法中调用 stateMachine.Initialize(idleState)，使玩家默认从待机状态进入游戏，运行时 Player.Update 在非死亡且游戏未暂停的情况下调用当前状态的 Update 方法，从而把不同动作逻辑交给对应状态类处理。

其主要优点就是把玩家的行为拆分成一个个相对独立的状态，待机状态主要是检测输入并切换到移动、跳跃或者攻击，移动状态处理水平速度和动画参数，跳跃和空中状态处理竖直速度、落地检测和墙面检测，冲刺状态根据 dashSpeed、dashDuration 和 dashDir 短时间改变玩家速度；不同的状态之间通过 stateMachine.ChangeState 方法切换，减少在脚本中写大量的互相嵌套的输入判断。

玩家的动作要和动画、技能释放配合起来，项目使用 PlayerAnimationTriggers 接收动画事件，在普通攻击动画的关键帧上进行攻击检测，在飞剑动作中调用 SkillManager.instance.sword.CreateSword 来创建飞剑；玩家接剑时调用 CatchTheSword 方法，先将当前状态设为 catchSwordState，再销毁场景中所有的飞剑对象，播放接剑音效，因为动画事件和状态机配合良好，所以可以保证技能释放、攻击判定、动画表现基本上同步。

在 Player 脚本中又添加了 BlockActionInputFor、IsActionInputBlocked 两个方法，

在开始菜单点击后短时间内屏蔽攻击输入，减少玩家点击开始游戏时立即触发普通攻击的情况；Die 方法设置死亡标记、播放死亡音效、触发死亡动画、冻结刚体，玩家死亡后就不响应移动和攻击输入了，整体上，玩家控制模块由状态机、动画事件、刚体控制三个部分构成，可以满足毕业 Demo 中移动、战斗、技能释放的要求。

玩家控制实现时，状态与物理不是两套互相不相关的逻辑。状态类决定当前应该接受哪些输入，Entity 提供 SetVelocity 与 ZeroVelocity 等公共接口，再由 Rigidbody2D 呈现位移结果。冲刺阶段普通移动碰到墙面前面就停下来，发现不合适的时候会取消。因此虽然角色的动作变化很多，但是实际的执行入口还比较集中，后续调节速度、冲刺时间、墙面交互等都可以沿着明确的位置进行排查。

普通攻击与技能释放的时候同样要注意到表现的节奏问题，不能只重视输入，忽视表现节奏。项目把判断移至了动画的关键帧上，玩家看到攻击动作挥出的时候，攻击区域才会被真正检测到敌人；Q 键弹反也放在了玩家的基本战斗输入里处理，只有当普通骷髅攻击动画开启反制窗口之后，CounterAttack 状态才会命中并造成反击伤害，Boss 战仍然主要依靠移动、冲刺和技能来应对。飞剑进入瞄准状态是在投掷事件发生之后才产生一个对象。菜单按钮点击之后，系统会暂时屏蔽时间，防止因为时间的影响而造成鼠标点击被误判为攻击。动作游戏 Demo 的细节虽然不明显，但是会影响操作是否方便。

在游戏进行的过程中，展示玩家的移动、跳跃、冲刺和普通攻击等图片

图 4.1 玩家控制与基础场景运行效果

表 4.1 玩家主要状态及职责

状态名称	动画参数	主要职责
Idle / Move	Idle、Move	处理待机、水平移动和基础输入检测
Jump / Air	Jump	处理跳跃、空中下落、落地和墙面检测
Dash	Dash	根据方向和持续时间完成冲刺动作
Attack / CounterAttack	Attack、CounterAttack	播放普通攻击动画，并在骷髅开放窗口时处理 Q 键弹反判定
AimSword / CatchSword	AimSword、CatchSword	控制飞剑瞄准、投掷后的接剑流程
Blackhole	Jump	释放黑洞技能并等待技能控制器结束

4.2 角色属性与伤害判定实现

属性系统用事件驱动的方式把战斗数据和界面显示联系起来，CharacterStats 用来维护生命值和伤害数值，角色受到伤害之后会更新 currentHealth，然后通知血条刷新，血条脚本只关心显示比例，不直接参与伤害计算，攻击脚本只负责触发伤害，不直接修改 UI，这样就可以使数据的变化和界面的反馈保持同步，同时减少模块之间的耦合。

受击方向处理对动作反馈影响较大，如果玩家只按照敌人自身朝向进行击退，那么当玩家从敌人背后攻击时，敌人就会向玩家的方向移动，表现出来的会比较不

自然；因此项目在伤害方法中支持传入攻击来源的位置，根据攻击者和受击者的相对位置来计算击退方向，使普通攻击、飞剑和 Boss 攻击在反馈上更加一致，也更接近玩家的直觉。

为减少角色短时间内多次受到多发伤害的情况，在属性系统中仍然保留了无敌时间以及死亡状态的判定，当角色死亡之后会被设置为 isDead，之后的伤害和行为的更新也不会被死亡逻辑所影响，在 Boss 激光、黑洞的持续伤害、近战攻击同时存在的情况下更为重要，可以避免在同一帧或者短时间内重复扣血、重复播放死亡动画等问题的发生。

角色属性系统主要由 CharacterStats、PlayerStats 和 EnemyStats 构成，CharacterStats 中定义 maxHealth、currentHealth、damage、isDead 和 isInvincible 等字段，TakeDamage 方法负责扣除生命值，并通过 onHealthChanged 事件通知 UI 刷新；当 currentHealth 减少到 0 时，CharacterStats 会调用 Die 方法记录死亡状态，PlayerStats 和 EnemyStats 在此基础上分别服务于玩家和敌人，使不同角色能够拥有独立血量与伤害数值。

Entity 类主要负责角色的通用行为，包括刚体、动画、碰撞检测、受击处理等功能，damage 方法有多个重载方式，可以按照 Transform 获取攻击来源，也可以传入 Vector2 位置和伤害值，这样设计后，普通攻击、飞剑、黑洞、Boss 法术、激光都可以使用相同的受击入口，为了使击退方向更自然，项目根据伤害来源的位置来

计算受击方向，而不是直接按照敌人自身朝向后退，从而减少玩家从背后攻击时敌人反向靠近玩家的问题。

伤害判定主要依靠 Collider2D、Trigger 和动画事件来完成，玩家普通攻击会检测出攻击动画关键帧内存在的敌人，并调用 enemy.Damage(player.transform)，飞剑命中敌人或者地面后停止移动，插入目标并关闭碰撞，玩家再次按键时飞剑回到原处；Boss 法术和二阶段激光都是通过各自的控制器检测到玩家的碰撞并调用 player.Damage，这样可以保证伤害来源和受击对象之间的关系清楚，也可以为以后添加新的攻击方式做好铺垫。

血量 UI 依靠 CharacterStats.onHealthChanged 事件来更新 Fill Image 的 fillAmount，使用事件驱动的方法减少 UI 每帧主动查询角色生命值的情况，还可以减少不必要的耦合；当玩家死亡或者 Boss 死亡的时候，UI 流程脚本会根据生命值的状态弹出相应的菜单，使角色属性系统不仅仅影响战斗的结果，也会参与到整个游戏流程的控制中。

受击流程不是简单的去掉一个数值。伤害先被 Entity.Damage 引入，然后根据 CharacterStats 判定生命值和是否死亡，之后才会发出受击的闪烁、击退和声音反应；如果目标已经死亡，那么后面受击的动作就不应该重复进行。普通敌人、玩家、Boss 虽然外观、行为不同，但是这条公共流程可以复用，所以攻击来源增加之后，系统不需要为每一种伤害重新写一套死亡、血量处理逻辑。

击退方向的改变也是战斗反馈设计的一种思想。根据攻击来源的位置来确定水平受力的方向，玩家站在敌人左侧攻击时，敌人就会向右后退，站在敌人右侧攻击时，敌人就会向左后退；当两者横坐标几乎重合的时候，再使用朝向作为兜底。不同于单纯的敌人背后方向退走，它更加符合玩家所看到的碰撞关系，也不会使敌人靠向玩家。

表 4.2 伤害相关脚本职责

脚本	职责	说明
CharacterStats	维护生命值、伤害和死亡状态	提供 TakeDamage、IncreaseHealthBy 和 MakeInvincible 等方法
Entity	提供通用受击、击退和检测能力	玩家、普通敌人与 Boss 共享基础逻辑
PlayerAnimationTriggers	在攻击动画中触发伤害检测	保证攻击判定与动画帧同步
Sword_Skill_Controller	处理飞剑命中、插入和返回	命中敌人后调用 Enemy.Damage
DeathBringerPhaseLaser	处理二阶段激光伤害	预警结束后检测激光范围内玩家

4.3 技能系统实现

就目前的 Demo 来说，飞剑技能的实现体现了技能生命周期管理的思想，飞剑生成之前由 Sword_Skill 计算鼠标方向并显示轨迹点，飞剑生成之后由 Sword_Skill_Controller 接管运动、命中、插入、召回和销毁的过程；玩家对象通过 AssignNewSword 保存当前飞剑的引用，当飞剑还没有召回的时候再按右键就会触发召回，不会重复生成新的飞剑，这样可以减少场景中出现多个失控飞剑的情况。

飞剑命中后既要考虑视觉效果又要考虑碰撞，飞剑碰到敌人或者地面的时候会停下来，关闭碰撞体并挂接到目标或者停留在命中位置上，使它表现出插入的状态，召回之后再移动到玩家附近，在接近玩家的时候切换到接剑的状态，这样可以使技能的过程更加生动，也可以方便答辩时进行说明。

黑洞技能依靠范围控制和定时伤害相融合来达成，黑洞形成之后会依照最大尺寸慢慢变大，检测到范围内敌人之后，进入黑洞影响范围内的敌人会被暂时冻结或者被限制行动，在持续时间内受到多次伤害；项目没有直接把敌人持续吸到中心，而是用冻结处理，减少敌人和玩家重叠时出现左右抽搐的情况，体现出玩法效果和物理稳定性的折中。

技能系统由 Skill 基类、SkillManager 以及具体技能脚本组成，SkillManager 通过 GetComponent 获取 Dash_Skill、Sword_Skill 和 Blackhole_Skill 实例，玩家状态只需要通过 SkillManager 访问技能，不需要直接了解技能内部细节；这种结构把输入状态和技能生成逻辑分离，能够降低玩家脚本的复杂度。

飞剑技能由 Sword_Skill 和 Sword_Skill_Controller 共同实现，Sword_Skill 负责读取鼠标右键输入，通过 Camera.main.ScreenToWorldPoint 将鼠标屏幕坐标转换为世界坐标，再计算玩家到鼠标的方向作为发射方向；瞄准过程中，GenerateDots 生成轨迹点，DotsPosition 根据初速度、重力和时间计算抛物线位置，玩家在释放前可以看到飞剑的大致轨迹，释放时 CreateSword 实例化飞剑预制体，并调用 SetupSword 传

入方向、重力、玩家引用和返回速度。

飞剑控制器是飞剑发射后状态的变化，飞剑在初始化时会根据 SetupSword 的方向来设置速度，命中敌人或者地面之后会进入插入状态，停止刚体运动并关闭碰撞，从而减少持续触发伤害；当玩家再次按下右键召回飞剑的时候，控制器使飞剑向玩家移动，飞剑接近玩家的时候调用 player.CatchTheSword 切换到接剑状态并销毁飞剑对象，使飞剑技能不再只是一次性投射物，而具有了投掷、命中、插入、召回、接剑的完整的交互。

黑洞技能由 Blackhole_Skill 和 Blackhole_Skill_Controller 来实现，黑洞生成之后，控制器根据 maxSize 和 growSpeed 逐渐扩大自身的范围，在持续时间内对进入范围内的敌人造成多次伤害；敌人进入黑洞触发器之后会被加入到 enemies 列表中，然后调用 enemy.FreezeTime(true)将敌人的行动冻结起来，离开或者黑洞结束的时候再通过 ReleaseEnemies 恢复敌人的状态，为了减少敌人在黑洞中心和玩家重叠时不断改变朝向的情况，黑洞逻辑选择冻结敌人而不是持续吸向中心，这样整体效果会更稳定。

飞剑技能的瞄准过程就有反馈作用，它是可视化的。Sword_Skill 把鼠标屏幕坐标转为世界坐标，根据玩家的位置来计算出发射的方向，轨迹点位置加上初速度和重力项，得到近似抛物线。轨迹点不是最终的承诺，只是给玩家提供一个参考，因此当鼠标位置发生改变时，就会自动更新。本设计使投掷技能的比对方法和固定方

向的子弹更容易被人理解，在答辩中能够清楚的看到输入和技能方向的关系。

飞剑发射出去之后还需要有受控的生命周期。Player 对象只保存当前飞剑的引用，场景中已经存在飞剑的时候不会重新生成新的飞剑；飞剑命中敌人或者地面之后会停止运动并进入插入状态，召回阶段再向玩家移动，接近玩家之后切换到接剑状态并销毁对象。把投射物从一次性攻击的对象变成可以被回收的技能，从而避免重复的实例化造成了画面和判定上的混乱。

黑洞技能用另一种方式来处理。控制器会记录进入范围的 Enemy 列表，根据持续时间计算伤害间隔，设定次数没有消耗完的时候就逐次调用 TakeBlackholeDamage；范围结束、敌人离开或者对象销毁的时候，控制器都会释放被冻结的敌人，防止技能结束之后 AI 还在冻结状态中。这里并没有一味地追求强吸附的效果，因为测试中发现敌人和玩家重合的时候会出现方向摇摆的现象，所以改为范围冻结和多段伤害。

技能冷却是以交互结果来计算的，而不是按键触发之后一起开始计时。飞剑投出后会停留在敌人或者地面上，只有玩家收回并接住它之后，冷却才会开始；黑洞在范围效果结束之后等待玩家重新落地，然后才进入冷却。经过这样的处理之后，技能动作、状态切换和界面提示就变得一致了，技能处于冷却状态的时候不会先播放上升的动作，再被脚本拦截回来。冷却显示 UI_SkillCooldownPanel 上获取的是技能剩餘百分比，利用图示指示灯以及向内圆弧遮罩的方式展示飞剑、黑洞可以继续

使用的状态。

黑洞有多段伤害的话，就会加大死亡线处理的难度。控制器只有在实际命中敌人时才会消耗一次伤害次数，当范围内有目标还在的时候，就会等待规定次数完成后再结束技能；如果最后一段伤害正好杀死敌人，那么循环就会先保存当前 Enemy 引用，再继续处理冻结和释放，防止碰撞体关闭之后敌人列表发生变化导致索引错误。被冻结的目标在死亡过程中还要恢复 Animator 的速度，否则死亡参数虽然已经写入，动画还是会停留在冻结帧上。

这里用游戏运行截图代替了飞剑轨迹、飞剑命中、黑洞技能效果等等

图 4.2 技能释放运行效果

表 4.3 技能实现流程

技能	触发方式	核心流程
Dash	LeftShift	检测冷却与方向，进入 Dash 状态并短时间改变速度
Sword	鼠标右键	显示轨迹点、投掷飞剑、命中插入、召回、接剑销毁
Blackhole	技能输入	生成黑洞、扩大范围、冻结敌人、造成多次伤害、缩小销毁

4.4 普通敌人 AI 实现

就目前 Demo 来说，普通敌人的检测逻辑主要是通过前方射线、距离判断以及战斗计时来实现的，当敌人看到玩家或者被玩家攻击的时候，就会进入战斗状态并刷新 battleTime，玩家离开检测范围之后，敌人不会立即停止，而是会在计时结束之

后退出战斗状态；这样既减少了敌人全图追踪，又避免了玩家刚离开检测边缘时敌人突然静止的突兀表现。

敌人朝向在敌人和玩家横向距离很接近的时候加入 targetOverlapEpsilon 做坐标容差，当敌人和玩家横向距离比较小时不会因为小位置差而反复旋转，从而保持现在的朝向或者直接进入攻击判定，使得普通的敌人动作更加稳定。

普通敌人死亡之后还要处理动画、碰撞、AI 状态，如果只播放死亡动画而不关闭交互，敌人就会继续挡住玩家或者继续触发攻击，如果直接销毁对象，又会失去死亡的表现，项目在死亡流程中综合处理状态切换、碰撞关闭、动画播放，使敌人既能看到死亡的反馈，又不会对后面的游戏过程造成影响。

普通敌人以 Enemy_Skeleton 为代表，Enemy_Skeleton 在 Awake 中创建 idleState、moveState、battleState、attackState 和 stunnedState，并在 Start 中初始化为 idleState；巡逻和待机状态用于敌人未发现玩家时的基础行为，battleState 用于发现玩家后的追踪和战斗判断，attackState 用于播放攻击动画并通过动画事件检测玩家，stunnedState 则用于受击后的短暂硬直。

敌人进入战斗状态的方式有看到玩家、被玩家攻击和受到黑洞伤害，Enemy_Skeleton 重写了 Damage 和 TakeBlackholeDamage 方法，在调用 base 逻辑之后会刷新战斗时间，使敌人在被攻击之后可以再次感知到玩家并进入战斗状态；这样就解决了敌人只在主动看到玩家的时候才会反应的问题，玩家主动攻击也会触发

敌人的战斗逻辑。

为了减少敌人出现不自然的抖动，在很多地方加入了容差和时间控制，当玩家和敌人横向距离很小的时候，敌人就不会频繁地改变自己的朝向；当敌人短时间内没有检测到玩家的时候，会经过 battleTime 之后退出战斗状态，而不是立即静止或者无限追踪，从而减少玩家站在敌人检测边界、坐标重叠或者射线断点附近时出现的左右抽搐问题。

普通敌人的 AI 是行为闭环型，在敌人处于待机或者巡逻的状态时，如果玩家靠近敌人的时候，会自动开启追踪模式，玩家与敌人发生碰撞的时候，敌人会发动攻击，被攻击过的敌人会在受到攻击之后再进行计时重新开始生命值恢复的进程，生命值全部清零之后才会触发死亡特效并且停止与玩家之间的交互；该种 AI 既有 Demo 的基本战斗体验又有为 Boss 战状态设计的结构。

4.5 关卡机关与精英敌人实现

关卡机关模块主要围绕落石陷阱和骷髅王通路解锁两部分展开，落石陷阱由 FallingStoneTrap 负责控制，脚本内部使用 Ready、Warning、Falling、Landed、Returning 和 Cooldown 等状态描述石块生命周期；Ready 表示机关可触发，Warning 通过轻微抖动提示玩家危险即将发生，Falling 根据加速度向下移动，Landed 保持短暂停留，Returning 缓慢回到初始位置，最后进入 Cooldown 以减少连续触发。

触发检测并没有直接放在石块本体上，而是为每一个石块创建了一个独立的

FallingStoneTrapSensor，传感器是一个触发器 Collider2D，只覆盖石块正下方一定的宽度范围，当玩家进入这个范围的时候调用对应的石块的 Trigger 方法；这样可以保证“谁检测到谁下落”，减少多个石块被同一个触发器连带触发，因为传感器固定在世界空间，石块下落和回收不会改变检测区域的位置，机关重复运行时更加稳定。

石块运动使用 Coroutine 逐帧推进，WarningRoutine 负责预警抖动，FallingRoutine 通过 currentFallSpeed 和 fallAcceleration 逐渐提高下落速度，并用 MoveTowards 靠近底部点，LandedRoutine 负责落地停留并持续检测玩家重叠，ReturningRoutine 则以 returnSpeed 缓慢回到初始位置；和直接使用刚体重力相比，这种脚本控制方式更容易保证每次下落距离、速度上限和回收位置一致，适合毕业 Demo 中需要稳定复现的机关效果。

落石伤害用接触即死的方式来处理，FallingStoneTrap 在下落和落地阶段通过 damageCollider.OverlapCollider 来检测玩家层级，如果检测到 Player，就关闭无敌状态并调用 TakeDamage 造成超过最大生命值的伤害；脚本中使用 killedPlayerThisDrop 标记减少同一次下落重复触发死亡，这样实现了机关危险性明确、便于测试的功能。

目前 Demo 中骷髅王不是重新编写出一套完全独立的 AI，而是对普通 Enemy_Skeleton 进行场景配置和参数增强，保留了普通骷髅的待机、移动、战斗、攻击、受击、死亡状态，生命值高、体型大、世界空间血条明显、头顶加金色扁平光环来区别于普通敌人；既复用了原有的状态机结构，又让玩家一眼看出这个敌人

和普通骷髅不一样。

骷髅王击败后的通路变化由 SkeletonKingTileGate 实现，该脚本持有骷髅王的 CharacterStats 引用、Ground Tilemap 引用和需要移除的 gateCells 数组，Awake 阶段缓存这些格子的瓦片、变换矩阵和颜色，OnEnable 阶段订阅 onHealthChanged 事件；当检测到骷髅王 currentHealth 归零时，OpenGate 遍历指定 Tilemap 坐标并执行 SetTile(cell, null)，由于被移除的是带碰撞的 Ground Tilemap 格子，视觉砖块消失后，TilemapCollider2D 也不会再阻挡玩家。

通路开启方案一开始是考虑半透明砖块的，但是实际演示中，这种提示很容易使玩家误以为砖块还在挡路，因此方案把 openedAlpha 设为 0，骷髅王被击败后，柱子底部三行砖块直接消失；这种方式的反馈更直接，碰撞与显示不会产生矛盾。

这里用游戏运行截图代替落石机关、骷髅王、击败后通路开启效果的描述

图 4.3 关卡机关与骷髅王解锁效果

表 4.4 关卡机关相关脚本职责

脚本或对象	核心职责	说明
FallingStoneTrap	控制石块预警、下落、停留、回收和冷却	通过状态枚举和 Coroutine 保证机关可重复运行
FallingStoneTrapSensor	检测玩家是否进入石块下方区域	每个传感器只触发对应石块，避免联动误触发
SkeletonKing	作为强化骷髅敌人承担关卡小 Boss	复用 Enemy_Skeleton 状态机，通过参数、血条和光环区分
SkeletonKingTileGate	监听骷髅王死亡并移除 Tilemap 砖块	通过 SetTile(null)同时取消显示和碰撞阻挡

落石机关的关键不是让石块简单地往下走，而是要使它在多次运行后保持一定的确定性，机关每次触发都会从缓存的 startPosition 出发，向 bottomPosition 移动，

结束之后又回到 startPosition；如果直接使用 Rigidbody2D 的重力，石块会因为碰撞、帧率、物理材质等因素而出现位置偏差，使用 Coroutine 和 MoveTowards 虽然物理真实感较弱，但是可以保证演示过程中每次下落的距离和回收的位置一致，对于答辩演示来说，稳定性复现比复杂的物理模拟更加重要。

预警阶段的轻微抖动起到反馈提示的作用，如果玩家的石块检测到玩家后立刻落下，玩家就会觉得伤害太突然，加入 warningTime 之后，玩家就可以看到石块即将运动，还有短暂的时间通过冲刺离开危险区域；这样的设计和 Boss 激光预警线的思路类似，两者都遵循了先提示、再造成伤害的原则，即使玩家失败，也能理解失败的原因，系统的表现比没有提示的瞬间伤害更加公平。

机关伤害用即死判定是为了演示的清晰度而做出的，普通敌人和 Boss 的攻击一般会造成固定的伤害，玩家可以通过血条来观察损失，落石属于场景危险物，如果只扣除少量的生命值，危险感就会降低，也不容易在演示中表现出机关的意义；项目通过 TakeDamage 传入超过最大生命值的数值来实现即死，在触发之前关闭玩家无敌状态，减少玩家刚受击后短暂无敌抵消机关伤害，同时 killedPlayerThisDrop 保证同一次下落只处理一次死亡，避免重复播放死亡流程。

骷髅王的设计重视的是基于已有敌人的强化，而不是重新创造复杂的敌人，它使用 Enemy_Skeleton 的状态机，所以待机、追踪、攻击、受击、死亡等行为与普通骷髅相同，只是数值、体型、血条、视觉标识不同；这样可以减少新增敌人带来的

脚本风险，也符合游戏设计中精英怪的常见做法，即用较少的机制变化和明显的视觉差异，使玩家感觉到它比普通敌人更特殊，头顶光环和较大的世界空间血条就是服务这种识别。

SkeletonKingTileGate 体现了事件驱动的思想，脚本不会在每一帧主动去遍历场景来寻找敌人是否存在，而是订阅 CharacterStats 的 onHealthChanged 事件，在血量发生变化的时候才进行检查；虽然目前项目规模不大，即使每帧都进行检查也不会造成明显的性能问题，但是事件驱动更符合模块解耦的思想，角色属性负责发出生命值变化的通知，关卡门只关心骷髅王是否已经死亡的结果，而不参与敌人的攻击、移动和动画细节。

Tilemap 开门时缓存原始瓦片也有维护的意义，目前最终效果是直接使砖块消失，但是脚本仍然保留了 cachedTiles、cachedTransforms 和 cachedColors，这是因为这样可以支持后续的扩展；如果希望通路开启时先闪烁、渐隐或者转移到无碰撞显示层，那么就可以复用缓存数据，而无需重新读取地图或者手动配置素材，当前 Demo 只使用最直接的消失效果，在可读性、演示的明确性以及开发成本之间做出了取舍。

就目前的 Demo 而言，机关和精英怪一起填补了 Boss 前面的过渡区，玩家通过落石时需要移动和冲刺，击败骷髅王时需要攻击和技能，之后通路打开才进入 Boss 区域；该流程使玩家在最终 Boss 前已经接触到了大部分的操作，不会出现所有系统都集中到 Boss 战才第一次使用的情况，对论文来说，也使得场景设计、敌人 AI、技

能系统和角色属性之间的联系更加紧密，而不是各自孤立存在。

4.6 Boss 战斗系统实现

Boss 战斗在普通敌人的基础上增加了节奏控制，Boss 不会一直追着玩家做单一攻击，而是根据攻击距离、冷却时间、当前阶段来决定行动；靠近玩家时，Boss 先尝试近战，距离较远或者条件满足时会释放法术，位置不合适时就会通过传送调整站位，这些行为使得 Boss 战比普通敌人战斗更具有变化。

传送逻辑要考虑到地面与障碍物，如果传到了地下或者墙壁里面的话，则会造成 Boss 无法进入战场中去，因此项目在传送位置的计算过程中是以玩家周围范围内的地面为候选区域，并且向下进行射线查找找到地面之后，再配合范围检测来判定该位置是否可用，从而使传送后能够更快地将 Boss 传送到玩家身边的战场上。

第二阶段转场是 Boss 战展示性的重要部分，当 Boss 的生命值小于某个阈值时，不会马上进入普通攻击的循环，而是切换到 phaseTransitionState，用激光预警线给玩家做出提示；预警线先锁定玩家的位置，短暂停顿之后变粗并造成伤害，连续几道激光结束之后再激活二阶段，这样就使得阶段的变化有了可见的反馈，也给玩家留出了通过移动或者冲刺躲避的机会。

Boss 死亡流程是 Demo 结束的条件，Boss 的生命值为 0 的时候会触发死亡标记，停止刚体移动，关闭碰撞，播放死亡动画，之后由 UI 流程显示通关界面；通关界面给出重新开始和退出两个选项，使 Demo 从开始菜单、战斗过程到结束菜单形成完

整的闭环，对于毕业设计演示来说，这样的闭环比单纯地出现一个敌人更能体现系统的完成度。

Boss 模块以 Enemy_DeathBringer 为核心，该类继承 Enemy，并在 Awake 中创建 idleState、battleState、attackState、spellCastState、phaseTransitionState、teleportState 和 deadState；Boss 默认处于 idleState，当玩家进入 bossFightDistance 范围后，ShouldStartBossFight 返回 true，Boss 开始进入战斗流程，相比普通敌人，Boss 状态更多、切换条件也更复杂，所以更能体现有限状态机对复杂行为的组织能力[3,10-11]。

Boss 的第一阶段主要是靠近玩家、近战攻击，为了减少 Boss 一直追到玩家身边之后就停下来被玩家打的情况，项目加入了攻击节奏控制，minAttackCooldown 和 maxAttackCooldown 用来随机生成攻击冷却，chaseBeforeAttackReadyTime 用来限制 Boss 只有在攻击即将可用的时候才会更加积极地追踪玩家；这样可以使得 Boss 在攻击间隔中有一定的停顿，减少始终贴着玩家移动造成的观感混乱。

Boss 二阶段由生命值阈值触发，当 stats.currentHealth 低于最大生命值的 phaseTwoHealthPercent 比例时，CheckPhaseTwo 将 Boss 切换到 phaseTransitionState；转场过程中 Boss 进入无敌状态并生成多道 DeathBringerPhaseLaser，激光控制器先显示细红色瞄准线，等待 warningTime 后逐渐变宽并进入伤害阶段，由于瞄准线在发射前会短暂停止追踪玩家，玩家可以根据

提示通过冲刺躲避，战斗反馈比直接瞬发伤害更清楚。

第二阶段激活之后，Boss 会通过 ActivatePhaseTwo 来提升移动速度、缩短攻击冷却时间，并且可以施法、传送，CastSpell 根据玩家的位置和移动方向生成法术预制体，TeleportNearPlayer 调用 FindPosition 寻找合适的地点；传送位置不是简单的随机赋值，而是根据玩家位置、arena 范围、地面 Raycast 结果和 OverlapBox 空间检测共同决定的，这样可以减少传送到地底或者墙体中的情况，如果找到合法位置，Boss 就会移动到玩家附近但是保持 minTeleportDistanceFromPlayer 距离，使传送既有压迫感又不会直接重叠。

Boss 死亡时，Die 方法设置 isBossDead，播放 Boss 死亡音效，切换到 deadState，并冻结刚体、关闭碰撞体，PlayDeathAnimation 负责清理动画布尔值并播放 deathBringerExit 动画；Boss 生命值归零后，UI_BossClearMessage 在短暂延迟后显示通关菜单并暂停游戏，Boss 战结束反馈更加明确，整体 Boss 系统通过阶段变化、攻击冷却、法术、传送和通关 UI 构成 Demo 中最完整的战斗流程。

Boss 的开战条件也是范围控制。角色只有进入到 bossFightDistance 之后，Boss 才会从待机状态进入到战斗状态，这时玩家在远处处理普通敌人或者机关的时候，不会提前将最终的战斗拉入到当前的画面中。进入战斗后，Boss 并不每一帧就靠近玩家，攻击冷却、施法冷却、追踪时机三者一起决定下一步的动作。玩家可以观察到靠近、停顿、近战、施法、位移之间节奏的变化，Boss 也不会只有一种表现形式，

即一直追击。

第二阶段的激光更多地是用提示加判断的方式。DeathBringerPhaseLaser 使用 LineRenderer 画出一条红色的线段，预警时线宽较细，真正发射时再逐渐变宽；激光在发射前会先锁定一个方向，之后就不会一直跟踪玩家了，玩家看到提示后还有闪避的空间。伤害检测用线段方向一致的 OverlapBoxAll 来检测，检测区域长度覆盖整个画面，宽度随着激光发射阶段的变化而变化，视觉上被打中但是扣不上血或者看不清为什么受伤的地方尽量靠近，减少看见被打中的却扣不上血或无法看清为何受伤的落差。

传送行为处理的是一种稳定的状况。候选位置并不是简单的给出一个偏移值，而是首先根据当前能够搜索到的区域得到目标横坐标，再向下 Raycast 找到地面，利用 Boss 碰撞体底部偏移将角色放至可站立高度，最后用 OverlapBox 判断周围是否还有地形占据。多次尝试都没有找到合法的位置的话，脚本就会强制传送。保留这样一种方式可以使 Boss 掉入墙体里、掉入地下或者和玩家重叠等复杂多变的情况也能够被测试所涵盖。

在游戏运行的时候提示画面图示截图所示，BOSS 出现了近战、法术或二阶段激光的提示

图 4.4 Boss 战斗与二阶段攻击效果

表 4.5 Boss 主要状态及切换条件

状态	触发条件	主要行为
Idle	玩家未进入 Boss 范围	保持待机，等待战斗开始
Battle	玩家进入范围且 Boss 未死亡	面向玩家，判断追踪、攻击、施法或传送
Attack	玩家进入攻击距离且冷却结束	播放近战攻击动画并造成伤害
SpellCast	二阶段且法术冷却结束	在玩家附近生成法术攻击
Teleport	二阶段且满足传送概率和冷却	寻找玩家附近合法地面位置并传送
PhaseTransition	生命值低于阈值	进入无敌并生成多道激光提示和攻击
Dead	生命值归零	播放死亡动画，关闭碰撞，触发通关菜单

Boss 战的节奏控制体现在攻击冷却、追踪距离上，早期如果 Boss 一直追着玩家移动，即使攻击冷没有结束，也会出现 Boss 贴在玩家身边来回跑动的情况，观感上就像是不能正确进入攻击状态；因此项目把 Boss 在攻击冷却快要结束的时候再更积极地追踪玩家，在距离较近的时候先切换到 Idle 或者攻击判断，这样 Boss 就不会一直挤压玩家的位置，也能让玩家看清 Boss 准备攻击的节奏。

第二阶段激光的实现要处理瞄准、公平性、伤害范围这三个因素的关系，激光预警线会先指向玩家的位置，但是发射之前会短暂停止追踪，减少玩家无论怎样移动都受到锁定命中的情况；发射阶段再逐渐变粗，用与视觉宽度相近的碰撞判定检测玩家，既能保证激光看起来横穿屏幕、有足够压迫感，又能给玩家通过冲刺躲避的机会，如果判定线过细，玩家会觉得明明被画面覆盖却没有受伤，如果预警阶段也持续锁定，玩家又会觉得无法躲避，所以最终方案是在视觉和可玩性之间做出折

中。

Boss 传送逻辑的稳定也会对战斗体验产生影响，传送位置如果只是用玩家坐标做简单的偏移，就会落在地面以下、墙体内部或者和玩家重叠的地方；项目通过向下 Raycast 找到地面，再用 OverlapBox 检测候选空间是否可以站立，使 Boss 尽量在距离玩家最近的位置出现，如果传送失败则不强行移动，而在原位置继续执行其他的动作，从而减少由于一次找点失败导致 Boss 卡死或者离开战斗区域的情况。

Boss 和黑洞之间还做了一定的约束。攻击、施法、传送动作如果完全保持原状，那么黑洞命中就会因为传送无敌或者状态占用而显得收益不足；目前实现中，Boss 被黑洞捕获的时候会打断这三类动作，回到原地停住的 Idle，再由公共伤害流程扣除生命值并给出受击闪白反馈。第二阶段的激光转场依旧有无敌保护，不会因为阶段演出被范围技能打断。试玩节奏下 Boss 的基础移动速度变快了，于是追踪阶段的压迫感就更加明显了。

4.7 UI 与音效系统实现

UI 系统采用集中式的管理方式来降低场景配置的成本，在开始菜单、暂停菜单、死亡菜单、通关菜单等场景的 UI 对象中使用 UI_BossClearMessage 进行创建或控制，当需要创建某个 UI 元素的时候，脚本会根据当前脚本执行环境创建该 UI 元素，并将 UI_BossClearMessage 的对象注册到当前脚本中去，保证场景运行时 UI 对象能够正常使用，如果场景中没有对应 UI 元素，也可以使用脚本来实现 UI 元素的创建。

菜单和游戏时间的关系用 Time.timeScale 来控制，开始界面、暂停界面、结束界面显示时 timeScale 设为 0，玩家和敌人不再继续行动，恢复游戏或者重新开始时再恢复为 1；需要注意的是，菜单按钮本身仍然可以响应，所以 UI 动画或者输入逻辑不应该依赖普通的 game time，这样才能保证暂停和结束菜单有实际的控制效果。

音效系统使用 AudioManager 统一加载并播放 Resources/Audio/SFX 目录下所有的音频文件，攻击、受击、死亡、飞剑投掷、接剑、Boss 法术、激光、菜单点击等动作都是通过枚举类型调用 PlaySFX 来实现；统一入口方便以后修改全局音量、单个音效倍率、重复播放的冷却时间，减少各个脚本中单独设置 AudioSource 造成的管理混乱。

UI 系统大致包括血条显示、开始菜单、暂停菜单、死亡菜单和通关菜单，UI_HealthBar 通过绑定 CharacterStats 和 Image 组件实现血量显示，当角色生命值变化时，根据 currentHealth 和 maxHealth 计算 fillAmount；UI_WorldHealthBar 用来显示世界空间中的敌人血条，使普通敌人受击反馈更加直观。

界面提示部分把提示信息放在玩家初始区域上方的背景处，提示内容包含移动、跳跃、冲刺、普通攻击、对骷髅使用 Q 键弹反、飞剑和黑洞按键，在开始试玩一段时间之后才会消失，不会压在角色身上也不会在第一次尝试之前就看到全部内容。敌方世界空间血条按照试玩情况变宽变厚，普通骷髅、骷髅王、Boss 受到攻击后生命变化更容易被发现。

菜单流程由 UI_BossClearMessage 集中管理，Awake 阶段调用 EnsureEventSystem 和 EnsureMenus，保证场景中有 EventSystem，创建必要的菜单对象，Start 阶段根据 PlayerPrefs 中的 SkipStartMenuKey 判断是否跳过开始菜单；重新开始时可以直接进入游戏，而不能再次停留在主菜单上，运行时按下 Esc 可以打开暂停菜单，再次按下 Esc 或者点击继续按钮即可恢复游戏，玩家死亡或者 Boss 被击败时会分别出现失败或者通关的菜单。

就目前的 Demo 来说，暂停和结束流程主要是通过 Time.timeScale 来控制的，当开始菜单或者暂停菜单出现的时候，游戏的时间被设置为 0，玩家、敌人、Boss 的状态更新都会停止，恢复游戏之后再将 Time.timeScale 恢复为 1；为了减少鼠标点击开始按钮导致玩家攻击的情况，开始游戏时会调用玩家的 BlockActionInputFor 短时间内屏蔽动作输入，虽然这些细节并不大，但是可以提高 Demo 演示时的自然度。



图 4.5 菜单与流程控制界面

音效系统由 AudioManager 来管理，AudioManager 采用单例的方式存在，它会从 Audio/SFX 目录中加载音频资源；玩家攻击、玩家受击、飞剑投掷、飞剑接回、敌人死亡、Boss 法术、Boss 激光、菜单点击等操作都会调用 AudioManager.PlaySFX 播放音效，音效不是核心算法，但是可以增加攻击、受击和菜单操作的即时反馈，提高演示完成度。

菜单脚本除了显示按钮外，在保持一局 Demo 的流程状态。开始菜单会把游戏时间暂停，按下开始键后恢复 Time.timeScale；Esc 打开暂停界面的时候，继续按钮、重新开始、退出按钮会根据当前状态来决定是否显示；玩家死亡或者 Boss 被击败之后，界面会延迟一小段时间真实时间再弹出，使死亡动画或者 Boss 结束反馈有机会先被看到。重新开始的时候采用场景重载的方式，在下一次的跳转中直接越过开始菜单，玩家从失败后可以直接进入到操作流程之中，而不需要去点击开屏按钮。

AudioManager 把音效资源名和枚举类型对应起来，在播放的时候只需要知道当前处在攻击、受击、飞剑、Boss 激光或者菜单点击等状态，就不用在各个脚本里面去多次加载文件。管理器将 AudioClip 缓存起来，设置在声音重复使用的时候比较短的冷却时间，从而保证短时间内不能出现太多的过于激烈的声音叠加情况。单次调用可以提供音量倍率，因此 Boss 激光、死亡音效这些音响效果比较强的素材可以通过单独减少来使用。音效系统规模小，但是通过这个层的统一入口来控制，调试更加方便。

5 系统测试

5.1 测试环境与测试方法

测试方法分为功能验证和回归验证，功能验证主要是看新功能是否按预期运行，飞剑可以投掷、召回，黑洞可以造成多次伤害，Boss 进入二阶段也属于该类检查；回归验证主要是看修改后的程序对旧流程的影响，调整 Boss 攻击距离之后还需要重新检查法术、传送、死亡菜单等，本设计为毕业 Demo，测试重点放在核心流程的稳定性以及现场演示上。

测试时要考虑 Unity 编辑器和 Windows 打包程序，编辑器上会出现 Console 错误、对象层次结构、组件引用等信息，在出现空引用、碰撞层级、动画参数类问题的时候能够及时发现和解决；打包程序更接近最终演示环境，可以从播放视频到退出，从加载场景到重载等各个功能点进行检查，避免因为临时测试环境而造成的无法演示的问题。

在论文说明中，系统测试的目标就是检验 Demo 的主要流程能否稳定运行，重点检查玩家控制、普通敌人 AI、Boss 战、技能、UI 菜单、音效、Windows 打包程序；测试环境为 Windows 系统，开发工具为 Unity 2022.3 LTS，测试方式以黑盒功能测试和开发过程中回归测试为主，由于该项目为毕业设计 Demo，且未进行大规模并发测试或者服务器压力测试，所以测试章节主要围绕单机运行流程展开。

测试分为编辑器测试和打包测试两部分，编辑器测试主要是对脚本的逻辑、动画的触发、碰撞的判断等进行测试，打包测试主要是检验在 Unity 编辑器中脱离出来后能否正常启动、进入游戏、暂停、重新开始、退出；每次对 Boss、技能、UI 流程进行修改之后，都会重新执行关键路径，防止某一个局部的修补造成整个演示受影响。

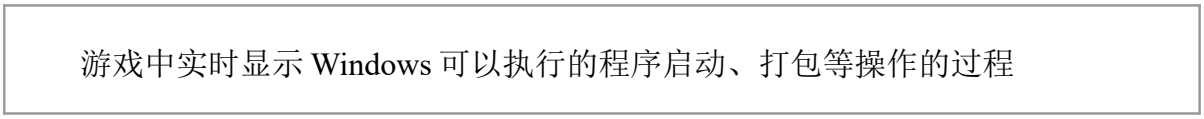


图 5.1 Windows 打包运行测试界面

关卡机关是后面测试的重点，测试时要从左右两个方向穿过落石区，主要看传感器只会触发玩家正下方的石块；石块下落、停留、回收、冷却等流程也要做重复检查，如果一个石块带动其它石块下落，一般说明传感器范围或者引用配置有问题，这时需要回到场景中检查 FallingStoneTrapSensor 的绑定。

骷髅王和通路解锁测试是对于事件链的全面检验，测试时先和骷髅王进行战斗，观察骷髅王的生命值、光环、受击反馈，当其生命值降到 0 时，检查右侧左边柱子下三行砖块是否消失，最后让玩家穿过原来的挡着的位置，这个测试中包含了很多 CharacterStats 事件、SkeletonKingTileGate 脚本、Tilemap 格子移除、碰撞更新等等。

5.2 功能测试

功能测试包括从游戏启动到 Boss 被击倒的所有过程，测试人员从开始菜单进入游戏后，首先保证玩家不会因为点击按钮立即对敌人发起攻击，接着测试移动、跳

跃、冲刺、墙跳、普通攻击和 Q 键弹反，再和骷髅敌人互动，观察敌人是否能够发现玩家、追踪、攻击、受击、死亡，同时确认弹反只在骷髅反制窗口内生效；最后进入 Boss 区域，检查 Boss 一阶段近战、法术、传送、二阶段激光、死亡和通关菜单，确认 Boss 攻击不走普通骷髅的弹反流程，该路径基本涵盖了 Demo 答辩中需要展示的主要内容。

除了主流程之外，还要对一些边界情况进行测试，例如玩家和敌人横坐标重叠时，敌人是否会频繁翻转；飞剑命中敌人边缘时，是否会插在空气中；Boss 传送后是否会落在地面以下；激光从不同方向发射时，是否会都造成伤害；玩家死亡后是否可以正确暂停并显示死亡菜单，也属于重点测试内容。这些问题不会影响系统的架构，但是会影响演示的观感，因此也属于重点测试的内容。

功能测试不能只停留在场景中单个对象上点开查看，而应该按照玩家的正常路线来测试。一次完整的测试应当从开屏开始，然后逐个测试相机跟随、角色移动、普通攻击、小兵、机关、骷髅王区域、Boss 战、死亡、通关等分支。只有对 Boss 脚本进行测试才能出现开始菜单输入屏蔽、Tilemap 门解锁、场景重开后时间倍率恢复的现象被忽视的情况。就目前的作品而言，路径测试比孤立测试更接近最终的答辩环境。

边界场景是用来检验功能在临界条件下能否有合理的表现出的。玩家从落石区域左右两侧经过时，传感器不应该提前触发过远的石块；敌人和玩家横坐标接近的

时候，敌人不应该连续翻面；Boss 激光从画面两侧发射的时候，整段射线都要有伤害判定；Boss 传送失败的时候，也不应该让战斗状态停滞。将现象写入测试中，就说明系统改进不是为了理想情况而做的，而是来源于运行过程中出现的不足之处。

在技能和 boss 联动回归测试的时候需要重视临界时机，飞剑接回前不能提前出现冷却，黑洞结束时玩家落地后就开始计时，boss 处于攻击、施法或者传送动作时被黑洞覆盖，仍然应该扣血；另外还要观察黑洞最后一段伤害击杀普通骷髅时死亡动画、碰撞释放是否连贯执行。把以上提到的检查项目加上到主流程回归后，就能抑制局部修复只对某一个场景有效的情况出现。

根据系统的功能要求，本文对系统的主要功能进行了测试，测试结果如表 5.1 所示。

表 5.1 系统功能测试表

测试项	测试内容	预期结果	测试结果
开始菜单	启动程序后点击开始游戏	菜单隐藏，游戏开始，玩家不立即攻击	通过
玩家移动	按方向键、跳跃键、冲刺键	角色能移动、跳跃、冲刺并播放对应动画	通过
普通攻击与弹反	靠近敌人执行普通攻击，并在骷髅开放反制窗口时按 Q	普通攻击产生受击和击退；骷髅可被弹反，Boss 不进入该弹反流程	通过
飞剑技能	瞄准、投掷、命中、召回飞剑	轨迹点显示，飞剑可插入敌人并返回玩家	通过
黑洞技能	释放黑洞并让敌人进入范围	敌人冻结并受到多次伤害，结束后恢复	通过
普通敌人 AI	靠近、离开、攻击敌	敌人能发现、追踪、	通过

	人	攻击、脱战和死亡	
Boss 一阶段	进入 Boss 范围并战斗	Boss 启动战斗、靠近玩家并执行近战攻击	通过
Boss 二阶段	将 Boss 血量打到阈值以下	Boss 进入转场，生成激光提示和伤害判定	通过
暂停菜单	游戏中按 Esc	游戏暂停，显示继续、重开、退出按钮	通过
死亡/通关菜单	玩家死亡或 Boss 死亡	显示对应菜单并暂停游戏	通过
打包运行	运行 Windows 可执行程序	可脱离 Unity 编辑器完成主要流程	通过
落石机关	从左右方向进入石块下方检测区域	只有对应石块下落，玩家被砸中后死亡，石块随后回收	通过
骷髅王战斗	攻击强化骷髅并观察血条、体型和死亡	骷髅王具有更高血量，死亡后不再阻挡流程	通过
通路解锁	击败骷髅王后靠近右侧柱子底部	柱子底部三行砖块消失，玩家可以继续前进	通过

5.3 典型问题与优化

第五类问题就是相机跟随范围不稳、画面构图不稳，项目使用 Cinemachine 控制主相机，直接移动 Main Camera 会被虚拟相机覆盖，所以需要调整 PlayerCamera 的 Orthographic Size、Screen Y、Dead Zone、Damping 等参数；经过调试之后，相机可以跟随玩家移动和跳跃，在画面下方不会出现背景色暴露的情况。

第六类问题就是音效音量不均衡，Boss 激光和死亡音效素材的响度较高，如果不使用 AudioManager 的 volumeScale 参数对单个音效进行倍率控制，在演示过程中

就会显得突兀；项目使用 AudioManager 的 volumeScale 参数对单个音效进行倍率控制，在不修改音频源文件的情况下压低了激光和死亡音效的音量，使音效反馈更加统一。

在开发测试阶段出现了几个典型的错误，空引用异常，如飞剑预制体缺少了 Sword_Skill_Controller 组件，SkillManager 不能获取到技能组件，UI 引用未绑定等都会导致运行时出现异常；对于这样的问题，在脚本里加上必要的空值检查以及 Debug.LogError 提示，在 Prefab 以及场景对象上附加组件引用，从而让错误定位变得更为方便。

第二类问题就是敌人和玩家位置接近的时候会出现抖动，普通敌人和 Boss 都会根据玩家相对位置翻转朝向，当两者横坐标几乎重合的时候，微小的位置变化就会导致朝向反复切换；为此项目加入了 targetOverlapEpsilon 等容差判断，在距离过小时不再翻转朝向，在敌人检测不到玩家之后通过 battleTime 延迟脱战，减少状态频繁进入和退出。

第三类问题是 Boss 的传送位置不稳定，直接用玩家的位置来设置传送坐标，Boss 就会传到地面以下或者被障碍物挡住；优化之后，Boss 会向下进行 Raycast 寻找地面，再用 OverlapBox 检查候选位置是否有足够的空间，最后结合 arena 边界和 minTeleportDistanceFromPlayer 限制位置，使传送更接近玩家但是不会与玩家或者地形重叠。

第四类问题就是技能视觉与判定不符，飞剑碰撞框过大就会产生插在空气中的感觉，黑洞持续吸引敌人到中心就会造成敌人和玩家重叠时左右抽搐；对于飞剑项目调小碰撞判定并使命中后关闭碰撞，对于黑洞项目取消持续吸向中心，改为冻结敌人并按时间造成伤害，使视觉表现和判定更稳定。

第七类问题就是机关触发区域过大或者多个石块联动，如果一开始用较宽的检测区域，玩家还没有走到石块正下方就可能会提前触发，或者一个触发器带动多个石块下落；最终将检测区域缩小到石块宽度附近，给每个石块单独绑定传感器，使玩家经过哪块石头下方就只触发哪块石头。

第八类问题出现在通路解锁提示上，如果玩家击败骷髅王之后只让砖块半透明，那么玩家就会认为这里不能走，如果玩家只关闭了碰撞但是保留了实体显示，那么就会出现视觉和物理不一致的情况；最后处理方式就是移除 Ground Tilemap 中的目标格子，使显示和碰撞一起消失，反馈也更加清晰。

表 5.2 典型问题与处理方式

问题	原因	处理方式
空引用异常	Prefab 或场景对象缺少脚本和引用	补全组件绑定，增加空值判断和错误提示
敌人重叠抖动	横向距离过小时反复翻转朝向	加入坐标容差，距离过小时保持当前朝向
Boss 传送到地形内	传送点未检查地面和空间	通过 Raycast 找地面，OverlapBox 检查可站立空间
飞剑插空	碰撞框过大或命中后仍继续检测	调小碰撞范围，命中后停止刚体并关闭碰撞

菜单点击触发攻击	开始按钮点击和攻击输入共用鼠标事件	开始游戏后短时间屏蔽动作输入
落石提前触发或联动	传感器范围过大或多个石块共用触发器	缩小检测区域，每个石块绑定独立 FallingStoneTrapSensor
通路开启不清晰	半透明砖块容易被误认为仍然阻挡	击败骷髅王后直接移除目标 Tilemap 格子

第九类问题就是新增内容之间会存在连锁影响，例如骷髅王体型的增大，攻击范围、受击范围、血条位置都要重新观察，落石机关加入即死判定之后，死亡菜单也不能只在敌人攻击造成死亡的时候才生效，在 Unity 项目中很多功能都是依靠场景对象引用来实现的，新增一个机关或者敌人之后，不能只测试它本身，还要沿着玩家死亡、场景重开、菜单恢复、再次触发机关等路径进行回归验证。

第十类问题是视觉反馈和程序判定容易不一致，在开发过程中出现过飞剑插入空气、Boss 激光视觉范围和伤害范围不一致、骷髅王通路半透明但是玩家不知道是否可以通行等现象；处理这类问题时，不能只从代码正确性出发，还要从玩家的观察角度来判断，最终项目尽量使视觉和判定一致，飞剑碰撞框缩小，激光发射宽度和伤害宽度接近，Tilemap 门开启时直接移除砖块，这些优化虽然不一定改变核心功能，但是会明显影响演示观感。

第十一条类问题演示用 Demo 的复杂度控制在合理范围内，随着功能的增加，项目中已经包含了玩家状态机、普通敌人、技能、机关、精英怪、Boss 以及 UI 流程等，如果再加入背包、钥匙数量、存档、多场景切换等功能，短期内会增加大量的

测试压力；因此目前版本只实现了击败敌人后解锁道路的设计意图，并且没有实现完整的钥匙系统，取舍是为了保持“击败敌人后解锁道路”的设计思路，同时减少由于一个演示节点而引入过多的系统。

这些优化过程也说明，小型游戏 Demo 的完成度不仅仅看功能多少来衡量的。新功能的加入会对动画、碰撞、数值、层级、UI、声音等等各方面都产生影响，骷髅王变大的光环以及血条位置需要重新安排，激光变粗的时候预警线、伤害宽度和音效音量也得一并考虑进去。论文如果只写最终结果，就不能看到这些实现的工作，所以在测试、问题分析等环节中保留重要的取舍，更能体现项目开发的过程。

目前版本还有进一步发展的余地。落石机关已经具有了可以重复触发、单块检测的特点，如果再加个关卡，那么根据场景需求可以调整预警时间、排列节奏；骷髅王通路解锁已经接收到生命值事件，以后还可以换成钥匙、机关按钮或者更完整的关卡事件管理。Boss 第二阶段已经具有阶段切换、预警攻击、结束菜单等，再往后推进时，可以增加招式或者检查点，而不需要推翻现有的玩家控制与属性系统。

5.4 测试结果分析

从模块上来说，玩家控制、技能系统、普通敌人 AI、Boss 战、UI 流程等模块之间已经形成了一条基本的闭环，玩家可以移动、使用技能进入战斗，敌人会根据玩家的行为做出相应的反应，Boss 会根据生命值进入第二阶段，UI 会在玩家死亡或者 Boss 被击败之后给出明确的结果；各个模块虽然还有一定的拓展空间，但是已经

可以支持毕业设计的系统说明和现场演示。

从维护角度来说，项目后续扩展的方向比较清楚，如果再增加敌人类型，可以复用 Enemy、EnemyStateMachine、CharacterStats，如果再增加技能，可以在 Skill 基类和 SkillManager 的基础上扩展新的技能脚本，如果扩大关卡规模，可以在现有的 Tilemap 和场景流程基础上增加新的区域、门锁、检查点；目前 Demo 保留了这些扩展入口，所以论文可以把其作为后续改进的方向，而不必夸大当前版本的完成度。

从测试结果可知，系统已经可以实现毕业 Demo 所需要的主要流程，玩家可以从开始菜单进入游戏，完成移动、跳跃、冲刺、普通攻击、技能释放等操作，普通敌人可以产生基本的 AI 反应，Boss 可以完成一阶段、二阶段以及死亡流程，UI 菜单可以覆盖开始、暂停、失败、通关等场景，打包程序也可以脱离 Unity 编辑器运行。

系统存在以下不足，一是关卡规模较小，还没有形成完整的大型非线性地图，二是背包、完整存档、复杂能力解锁系统没有被当前 Demo 重点实现，三是敌人种类和技能数量较少，战斗丰富度有待提高；这些不足不会影响当前毕业 Demo 的完整演示，但是可以作为后续扩展的方向。

就整体而言，项目稳定、可演示，玩家控制、敌人 AI、Boss 战、UI 流程可以形成闭环，也可以支持后面论文的分析。

测试结果说明，系统的重点已经由原来单个脚本就可以运行变为多个模块之间

的衔接是否通畅。Windows 打包程序可以启动，开始菜单可以正确地暂停场景，玩家进入之后不会误触攻击，战斗中血条、音效会随着伤害的变化而变化，失败和通关都可以进入对应的菜单，这些表现共同说明 Demo 已经具备了基本的发布形态。只在编辑器中查看某个状态的切换不能包含这层流程的结果。

与此同时，当前测试结果仍然在可以验证的范围之内。项目证明了单机关卡流程、角色状态、交互反馈，但是没有做并发性能测试、联网同步、长期存档可靠性的测试，所以测试分析只对现有的 Demo 进行结论。将范围说得越具体越好，可以使得后面改进的方向和已经完成的内容相脱离，防止论文把尚未实现的能力当作已有的成果来写。

打包验证还要做和编辑器检查不一样的工作。编辑器能调取 Console、组件引用、场景层次这些信息来修改碰撞、动画之类的，定位比较方便；可以执行的程序更像实际演示环境，对于问题的识别来说具有比较明显的价值，开屏、退出按钮、重开环节、音量反馈是否正常、窗口是否运行等都可以得到较为详细的排查结果。使用两者相结合的方式可以找出开发过程中出现的脚本错误，并且能够保证答辩时不能只有 Unity 编辑器运行的功能而不能在 Unity 编辑器之外使用资源或者菜单的情况出现。

所以在测试章节里保存重要的运行截图和菜单节点会更加有说服力。玩家控制画面、机关区域、Boss 二阶段提示、通关界面分别对应不同的模块，截图既不是装

饰也不是单纯的展示，而是可以体现论文所描述的交互已经在同一个项目中运行了。

后续补图只需要截取这些真实的节点，就能和正文的叙述形成对应的关系。

结　论

项目在完成的过程中经历了多次问题修复、功能收敛、关卡流程的完善，早期的问题主要是技能预制体引用、碰撞判定、空引用异常等，中期主要是普通敌人和 Boss 的状态切换、检测范围、受击反馈等，后期主要是菜单流程、Boss 二阶段提示、落石机关、骷髅王通路解锁、音效平衡、Windows 打包等；经过这些调整，系统逐渐由一个角色控制原型变成了包含开始、战斗、结束流程的可运行 Demo。

本课题研究目的在于将 Unity2D 开发中常用到的 Tilemap 场景、Rigidbody2D 物理、Animator 动画、有限状态机、碰撞伤害、UI 菜单、音效加载、可运行程序发布等技术点整理出来，成为一个完整的小项目。虽然小但是可以表现出完整的现代游戏 Demo 的从需求分析到测试验证的过程。

本文以 Unity 引擎、C#语言为基础，设计并实现了一个 2D 类恶魔城动作游戏 Demo，项目以毕业设计可运行、可演示、可说明为目标，实现了玩家控制、普通敌人 AI、Boss 战斗、技能系统、血量显示、菜单流程、音效管理、Windows 打包发布等功能，系统使用有限状态机组织玩家、敌人、Boss 的行为，把复杂的动作切换和 AI 逻辑拆分成各个状态，提高了代码的结构清晰度和后续的维护性。

在玩家控制部分，系统实现了移动、跳跃、冲刺、墙面交互、普通攻击、骷髅弹反、飞剑、黑洞技能等功能，在敌人 AI 部分，骷髅敌人可以完成巡逻、发现玩家、追踪、攻击、受击到死亡的流程，在关卡交互部分，落石机关可以根据玩家的位置

独立触发并循环回收，骷髅王被击败之后可以解除 Tilemap 通路，在 Boss 设计方面，DeathBringer 具备近战攻击、法术攻击、传送、二阶段激光转场和死亡反馈，Boss 攻击不包含在当前弹反范围内，在 UI 方面，系统实现了开始、暂停、死亡、通关菜单，并用血条显示角色的生命值，从测试结果来看，Demo 可以在 Windows 环境下独立运行并完成主要的战斗和关卡演示流程。

本轮完善之后，系统演示的闭环更加接近一个完整的关卡，玩家不再只是单纯地和敌人交换伤害，还要经过机关区域、识别精英敌人、战斗解锁道路、最终 Boss 战；这个过程虽然规模不大，但是已经包含了横版动作游戏常见的移动挑战、战斗挑战和流程门控，对于毕业设计来说，这些内容可以说明项目不仅实现了角色控制，还考虑到了关卡组织和玩家体验。

从技术收获的角度来说，项目加深了对于 Unity 组件协作方式的认识，Rigidbody2D 和 Collider2D 完成物理交互，Animator 完成动作表现，Tilemap 完成地图和碰撞基础，MonoBehaviour 脚本把输入、状态、事件、UI 联系起来；在开发过程中遇到的问题也说明，游戏项目难在多组件之间引用、时序、反馈是否一致，通过反复调试空引用、碰撞范围、相机边界、音效音量、菜单时间控制，项目渐渐形成较为稳定的运行状态。

后续可以再从三个方面进行完善，一是把骷髅王通路扩展成正式的钥匙或者机关门系统，使不同的敌人、道具、机关都能触发同一个 LevelEvent，二是增加更多

的关卡截图和运行记录，使论文中图 4.3、图 4.4、图 4.5 都是真实的场景，三是进一步调节敌人伤害、攻击冷却和玩家技能冷却，使 Demo 不仅可以通过，还可以更接近正式的动作游戏。

由于开发周期所限，本项目还存在一些不足，关卡规模小，没有完整的大型地图和复杂的门锁，存档和背包系统只是作为以后的方向保留，目前的 Demo 没有把它们完全展开；敌人类型和技能数量也较少，战斗平衡还需要进一步的调整，在现有的架构基础上增加地图区域、能力解锁、道具收集、更多的敌人类型以及正式存档系统，使项目从毕业 Demo 继续发展成为更加完善的 2D 动作游戏。

从测试覆盖角度来说，目前 Demo 的验证重点已经从单个功能是否可以运行，转向了多个功能能否连续运行，玩家可以从普通敌人开始，躲避落石机关，击败骷髅王后打开通路，最后进入 Boss 区域完成通关，这条路径包含输入、物理、动画、AI、Tilemap 事件、UI 菜单、音效等各个模块，如果这条路径可以稳定运行，那么项目就具备了比较完整的毕业演示价值。

从代码维护的角度来说，新增的机关并没有破坏原有的玩家、敌人和 Boss 脚本，而是单独放在 Scripts/Traps 目录下进行维护，落石只通过 Player 和 CharacterStats 来产生结果，骷髅王开门只监听 CharacterStats 并操作 Tilemap，两者都没有直接修改玩家状态机或者 Boss 状态机，边界清晰利于后续删除、替换或者扩展机关内容，也符合模块化设计的要求。

从作品完整性角度来讲，新增的内容加强了 Demo 的中段流程，原本玩家会直接进入 Boss 战，流程较短，加入机关和精英敌人之后，玩家在到达 Boss 之前需要完成一段可以展示的关卡挑战；这样一来，论文中关于需求分析、总体设计、详细实现和测试章节就多出了许多实际的对象来支撑，不会使论文只对主角和 Boss 这两个模块进行研究。

对于答辩展示来说，系统说明要对应到现场操作，目前的 Demo 可以按照固定的路线进行演示，从开始菜单进入游戏，展示玩家移动、跳跃、冲刺、普通攻击，然后与普通骷髅交互，说明敌人检测、追踪、攻击、死亡流程，接着通过落石机关，说明触发器、协程、即死判定，击败骷髅王后展示 Tilemap 通路开启，最后进入 Boss 战，展示一阶段攻击、传送、法术、二阶段激光和通关菜单；这条演示路线与论文结构基本一致，可以把抽象模块落到具体的画面中。

从代码阅读和后期维护的角度来说，项目仍然有比较清晰的目录边界，玩家行为在 Player 目录下，敌人和 Boss 在 Enemy 目录下，技能在 Skills 目录下，机关在 Traps 目录下，UI 和音效分别由 UI 和 Managers 目录来承担，新增落石和骷髅王通路之后，机关逻辑没有被写入 Player 或者 Boss 脚本中，而是通过少量引用和事件来完成协作，这样的组织方式有利于在答辩时说明模块职责，也方便后续发现机关问题时快速定位到对应的脚本。

如果后续再做为更完整的课程作品或者个人作品，可以把目前验证过的机制抽

象成可复用的配置，落石机关可以扩展为 Trap 基类，支持尖刺、移动平台、地刺等不同的危险物，骷髅王开门可以扩展为门锁组件，由敌人死亡、钥匙拾取或者机关按钮触发，Boss 激光也可以扩展为独立攻击预制体，供其他 Boss 复用；这样既能够保持目前 Demo 已经调通的稳定逻辑，又可以使得项目在后续扩展时减少重复编写相似代码。

参 考 文 献

[1] Bhosale T, Kulkarni S, Patankar S N. 2D Platformer Game in Unity Engine[J]. International Research Journal of Engineering and Technology, 2018, 5(4): 3021-3024.

[2] Pichlmair M, Fasterholdt M, Holmgård C. You say jump, I say how high- Operationalising the game feel of jumping[C]//Proceedings of DiGRA/FDG. 2016.

[3] Millington I. Artificial Intelligence for Games[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[4] Gregory J. 游戏引擎架构[M]. 叶劲峰, 等译. 北京: 电子工业出版社, 2014.

[5] Nystrom R. 游戏编程模式[M]. 侯金标, 译. 北京: 人民邮电出版社, 2015.

[6] Swink S. Game Feel: A Game Designer's Guide to Virtual Sensation[M]. Burlington: Morgan Kaufmann, 2008.

[7] Schell J. The Art of Game Design: A Book of Lenses[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[8] Unity Technologies. Unity User Manual 2022.3 LTS[EB/OL]. [2026-05-13]. https://docs.unity3d.com/2022.3/Documentation/Manual/index.html.

[9] Unity Technologies. Unity Scripting API 2022.3 LTS[EB/OL]. [2026-05-13]. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/.

[10] Bourg D M, Seemann G. AI for Game Developers[M]. Sebastopol: O'Reilly Media, 2004.

[11] Buckland M. Programming Game AI by Example[M]. Plano: Wordware Publishing, 2004.

[12] Rabin S. Introduction to Game Development[M]. 2nd ed. Boston: Cengage Learning, 2010.

[13] Adams E. Fundamentals of Game Design[M]. 3rd ed. Berkeley: New Riders, 2014.

[14] Trefry G. Casual Game Design: Designing Play for the Gamer in All of Us[M]. Burlington: Morgan Kaufmann, 2010.

[15] Microsoft. C# documentation[EB/OL]. [2026-05-13]. https://learn.microsoft.com/dotnet/csharp/.

致　　谢

就目前的 Demo 来说，本次毕业设计从选题、功能实现到论文撰写，都得到了指导老师的耐心指导和帮助，在项目开发的过程中，老师对课题方向、系统功能边界以及论文结构提出了重要的建议，使我在 Unity 项目开发和毕业论文写作之间找到了结合点，从而完成了从 Demo 实现到文档整理的过程。

同时，感谢在学习和开发过程中给与帮助的同学和朋友，在调试玩家控制、敌人 AI、Boss 战、UI 菜单等时，他们的试玩反馈使我发现了很多细节问题，也使我的 Demo 更加稳定、更加有演示价值。

还要感谢大学期间所有的授课老师，经过本次毕业设计，我对面向对象程序设计、Unity 组件化开发、有限状态机、游戏交互以及项目版本管理有更全面的认识，这些经历也会成为以后继续学习和实践的重要基础。